26 ОКТЯБРЯ 2026 / ДНЕВНИК КОМПИЛЯТОРА

Один день из жизни компилятора MinZ

Ruby и Crystal. Битовые потоки и идеи BEAM. Ленивые мелодии, генераторы и Z80.

Три иллюстрированных лонгрида

18 Mermaid-схем / реальные примеры / измеряемые результаты

MinZ project / при участии Astra

Содержание

01. Ruby рассказывает, Z80 считает 3

02. Мелодия между двумя файлами 29

03. Чемодан без виртуальной машины 53

В PDF сохранены векторные схемы. Широкие иллюстрации размещены на горизонтальных страницах A5; большие схемы при необходимости разделены на две части.

Срез main: a716ff184. CP/M-цепочка уже проверена. Spectrum - отдельный локальный эксперимент. Полного нативного MIDI -> VT2/VTX-конвертера пока нет.

Ruby умеет рассказывать. Z80 умеет считать. Как дать им общий язык

Crystal и Nanz, macro и meta, битовые схемы и идеи Erlang/OTP.

26 октября 2026 года. Первая часть дневника MinZ: Crystal, метапрограммирование, двоичные потоки и идеи Erlang. Срез основного репозитория - `a716ff184`.

В MIDI-файле нет нот в привычном музыканту смысле. Там встречаются сообщения: нажали клавишу, отпустили клавишу, поменяли инструмент, прошло некоторое количество тиков. Чтобы получить ноту с началом и концом, нужно связать два события. Чтобы получить строку трекера, нужно ещё перевести время в другую сетку. А затем выяснить, что делать с несколькими одновременно звучащими нотами.

На Ruby эту работу удобно описывать через объекты, коллекции и небольшие выразительные преобразования. На Z80 каждый такой объект сразу вызывает вопросы: сколько байтов он занимает, кто владеет памятью, переживёт ли ссылка следующий прочитанный сектор? Между этими способами думать есть немалое расстояние.

Сегодня в MinZ через него уже проходит вполне осязаемая цепочка. Две отдельные программы CP/M читают и записывают файлы: `PAIR.COM` превращает MIDI Chronos в **1485 парных нот**, а `FLATTEN.COM` получает из них **2153 события** трёх выходных каналов. Ноты и события сверены с исходным Ruby. Программы запускаются отдельными процессами эмулятора, поэтому между стадиями действительно остаётся файл, а не спрятанная таблица в памяти теста. Подробности и ограничения зафиксированы в доказательстве файловой цепочки.

До полного нативного MIDI→VT2/VTX-конвертера ещё остаются музыкальные стадии и конечный формат. Но уже полученный результат позволяет обсудить более интересный вопрос: какие свойства современных языков помогают маленькой машине - и где именно должна происходить работа, чтобы они ей помогали?

Красота Ruby здесь имеет практическую функцию

В проверяющем коде есть такой фрагмент. Это настоящий Ruby из oracle.rb, запускающего классы исходного Autosiril:

```
paired = track.events.select do |event|
  event.respond_to?(:note) && event.respond_to?(:off) && event.off
end
next if paired.empty?

paired.each do |event|
  note = VNote.new(event.note, event.time_from_start,
                   event.off.time_from_start, 15)
  notes << {start: note.start, off: note.off,
            pitch: note.note, source: source}
end
```

В нескольких строках видны предметные понятия: дорожка, событие, завершённая нота, время, источник. Такая запись хороша ещё и тем, что её удобно оставить независимым эталоном. Мы не переводим этот Ruby в Go, а потом не сравниваем две версии собственного пересказа. Проверка загружает исходные `Setup` и `VNote`, использует `midilib` и получает результат привычным для оригинала способом.

Это существенно для старого музыкального инструмента. Его поведение состоит из алгоритма и накопившихся особенностей. Например, при окончании дорожки незакрытые ноты в выбранном Ruby-пути получают

синтетические note-off с унаследованной последней дельтой времени. Для нескольких оставшихся нот эта дельта применяется последовательно. Соблазн «очевидно исправить» завершение на одну общую отметку времени привёл бы к другому результату.

Нативная стадия сохраняет эту особенность явно. Корпус проверяет обычное завершение, частично записанные окончания перед переполнением строки и переполнение синтетического 32-битного времени. Это хороший пример того, как читаемый высокоуровневый оригинал помогает низкоуровневой реализации: он сохраняет смысл, который легко потерять за масками, смещениями и границами массивов.

Ruby в этой истории исполняется на машине разработки. На Z80 работают скомпилированные Nanz и небольшие функции Crystal. Само это распределение обязанностей оказалось продуктивной частью архитектуры.

Crystal приносит уже принятое решение о типах

Crystal особенно интересен рядом с Ruby: знакомые `def`, блоки, выразительные методы, но решение о конкретных типах можно получить до исполнения целевой программы. Для MinZ было бы дорого заново реализовывать чужие перегрузки, вывод типов и правила раскрытия макросов. Поэтому исследовательский frontend использует библиотеку настоящего компилятора Crystal.

Путь проходит через `Crystal::Compiler` с отключённой генерацией машинного кода: нормализацию, семантический анализ и cleanup. Экспортёр обходит достижимые специализации вызовов, уже разрешённые Crystal, и переводит поддержанное подмножество в Lanz - текстовое представление для передачи в общий HIR. Это описано в итерации native frontend; точки входа находятся в native_frontend.cr и typed_export.cr.

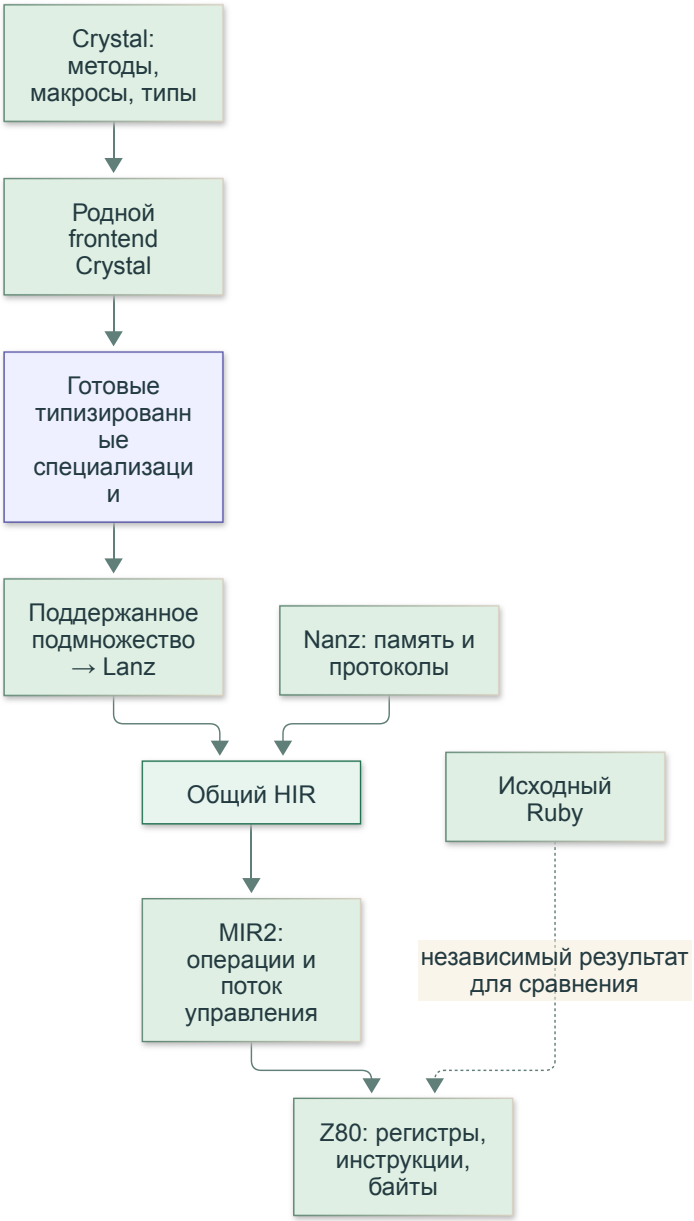


Схема 1.1 · На узком экране схему можно прокрутить горизонтально.

В specialize.cr достаточно написать:

```
def twice(x)
  x &+ x
end

def flavor(x : UInt8)
  x &+ 3_u8
end

def flavor(x : UInt16)
  x &+ 300_u16
end

def byte_demo(x : UInt8)
  twice(flavor(x))
end

def word_demo(x : UInt16)
  twice(flavor(x))
end

byte_demo(10_u8)
word_demo(1000_u16)
```

Crystal выбирает нужный `flavor` и создаёт конкретные варианты `twice` для `UInt8` и `UInt16`. Адаптер получает уже эти решения. Оператор `&+` здесь важен: он явно задаёт арифметику с переполнением по разрядности. Случайное превращение «обычного числа» в байт было бы скрытой сменой семантики.

Общий HIR позволяет разным frontend'ам пользоваться одной работой над вызовами, структурами, циклами и генерацией кода. Если проблема обнаруживается в общем понижении поля структуры, её исправление

может помочь и Nanz, и Crystal. Такие случаи уже случились в экспериментах с буферами и borrowed views.

Границы остаются конкретными. Это семейство проверяемых экспортёров и мостов в `research/crystal-hir`, а не обещание скомпилировать любой Crystal-проект для Z80. Возможности добавлялись отдельными итерациями: скаляры, записи и указатели, представления срезов, итерация, буферы, кольца, генераторы. Перенос всего объектного runtime, GC и произвольной стандартной библиотеки из этого не следует.

Маленький мост должен знать, что именно он соединяет

В музыкальной стадии Crystal отвечает, например, за перевод тика в строку. Вот реальная функция из music.cr:

```
def crystal_quantize(tick : UInt32, ppqn : UInt32,
                     per_beat : UInt32) : UInt32
  whole = (tick // ppqn) &* per_beat
  fraction = ((tick % ppqn) &* per_beat) &+ (ppqn // 2_u32)
  whole &+ (fraction // ppqn)
end
```

Она отделяет целые доли от остатка и округляет дробную часть. Nanz вызывает её через объявленный скалярный интерфейс. До вызова проверяются PPQN, число строк на долю и предельный tick: конечная строка должна поместиться в `u16`, а промежуточная `UInt32`-арифметика не должна незаметно обернуться. В pair.nanz предел вычисляется до квантизации:

```
tick_limit = (u32(65536) * ppqn - 1 - ppqn / 2) / per_beat
if tick > tick_limit { fault = 202
    return 0 }
return u16(crystal_quantize(tick,ppqn,per_beat))
```

Это фрагмент существующей функции `paired_row`; `ppqn`, `per_beat`, `tick_limit` и `fault` определены её окружающим кодом.

Проверка интерфейса тоже оказалась содержательной работой. Manifest утверждает, что функция принимает три `u32` и возвращает `u32`. Однако утверждение в JSON ещё ничего не гарантирует о настоящем теле: оно может иметь другую сигнатуру, неожиданные зависимости, операции с памятью или иной способ возвращать ошибку.

Особенно тонкая проблема обнаружилась с именами. Импортированное тело могло сослаться на неизвестный ему `fault`. После присоединения к Nanz-модулю имя вдруг находило одноимённую глобальную переменную. Так предполагаемая независимая функция получала доступ к чужому состоянию.

Теперь выбранный экспорт проверяется отдельно, до разрешения имён в принимающем окружении. VerifyClosedNames требует явных локальных объявлений и проверяет также адресные ссылки. Мост дополнительно проверяет настоящую ABI, эффекты и ABI ошибок. Тестовые мутации с чтением, записью и `(addr fault)` отвергаются. Проверки лежат рядом с MIDI runner.

Это небольшая, но важная граница доверия. Функция остаётся понятным компонентом, когда её зависимости перечислены в интерфейсе и действительно соответствуют телу. При этом общий режим проверки HIR сохраняет прежнюю совместимость Nanz; строгая проверка имён включается явно для таких мостов.

Две метапрограммы встречаются до запуска Z80

Crystal-макрос получает узлы синтаксического дерева при компиляции и порождает код. Это прямо описано в официальной документации macros. Nanz тоже умеет выполнять метафункции во время компиляции, в своём MIR2 VM, и строить типизированный HIR.

Отсюда возникла занятная возможность: написать удобное выражение на Crystal, а специализированное тело получить настоящим Nanz-генератором.

```
macro meta_repeat(value, count, delta)
  minz_meta_repeat({{value}}, {{count}}, {{delta}})
end

def three_iterations(x : UInt8)
  meta_repeat(x, 3_u8, 7_u8)
end
```

Это код из meta_showcase.cr. `minz_meta_repeat` - явный маркер согласованного протокола. Для нативного Crystal у него есть обычное тело с циклом: эталон по-прежнему можно исполнить независимо. Целевой импортёр распознаёт маркер и вызывает bridge, который запускает Nanz `fun @repeat`.

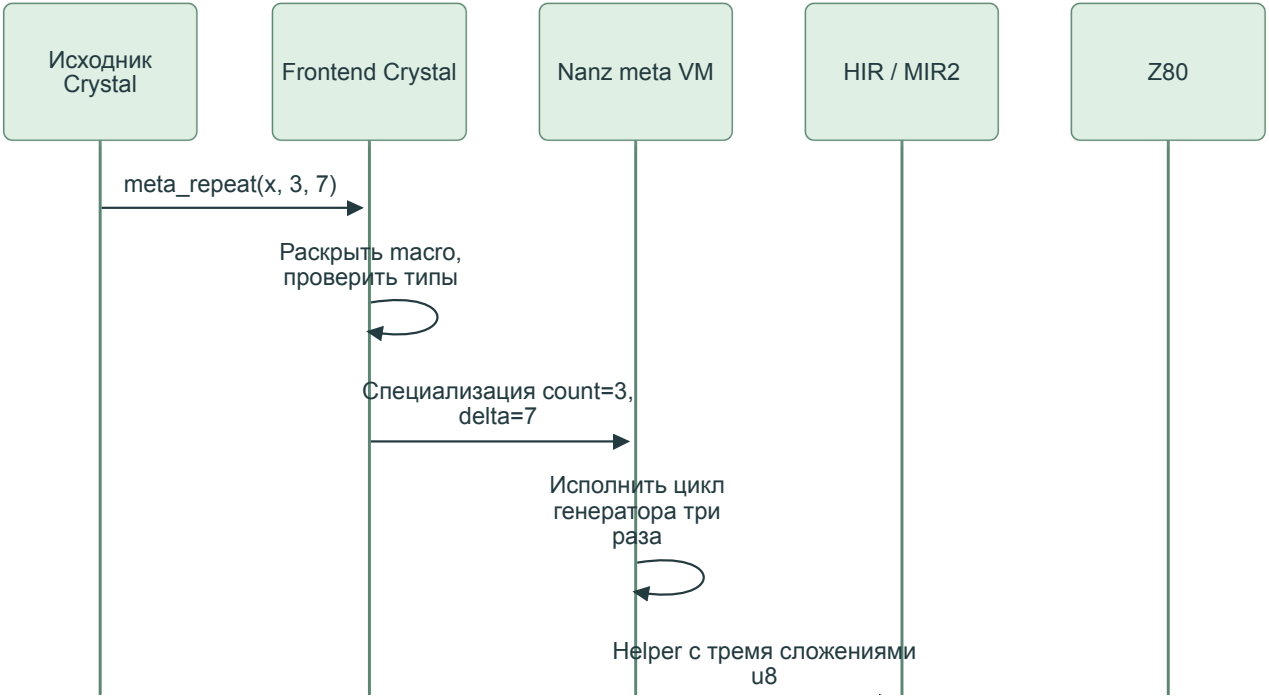


Схема 1.2 / часть 1 из 2

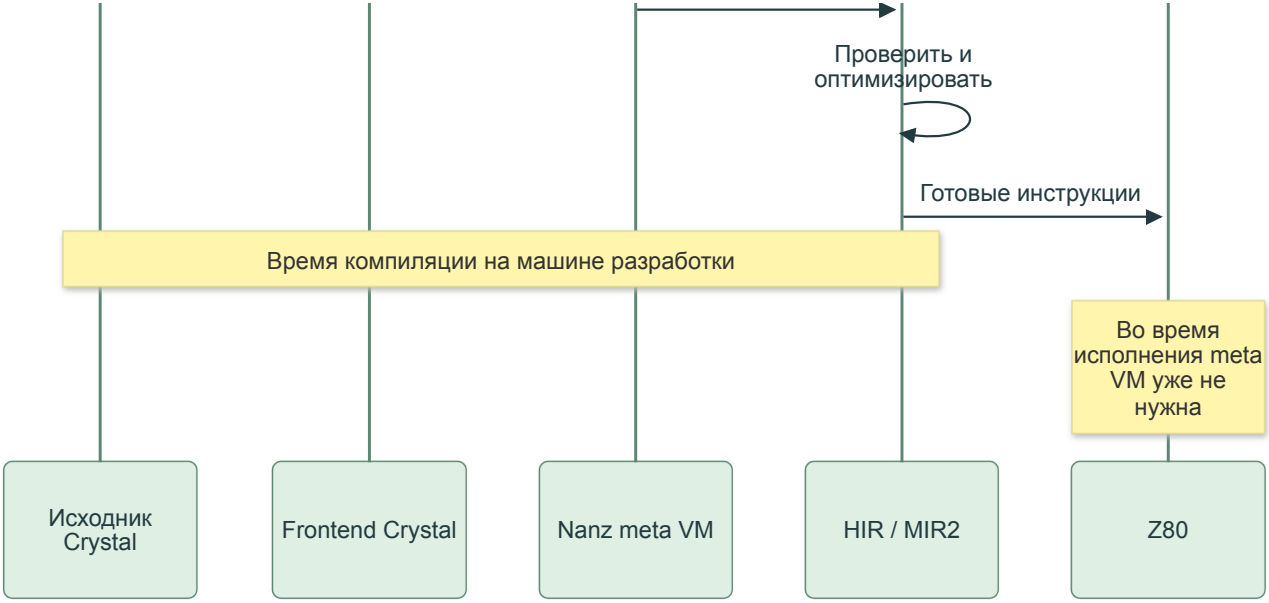


Схема 1.2 / часть 2 из 2

В repeat.nanz генератор действительно строит узлы HIR. После разбора compile-time аргументов его основная часть выглядит так:

```
let input: u16 = hir_caller_ref(c"value", c"u8")
let temp: u16 = hir_ref(c"meta_result", c"u8")
let block: u16 = hir_block()
hir_append(block, hir_var(c"meta_result", c"u8", input))
i = 0
while i < count {
    hir_append(block, hir_assign(temp,
        hir_binary(c"+", temp, hir_int(u16(delta), c"u8"))))
    i = i + 1
}
hir_append(block, hir_assign(input, temp))
emit_hir(block)
```

Цикл здесь исполняется компилятором. Для `count=3` он создаёт три сложения; дальнейшие проходы могут их упростить. По одному вызову такой системы рано делать вывод о выгоде любого макроса: разворачивание увеличивает размер кода, и на Z80 это тоже расход. Зато теперь выбор сделан в месте, где известны число повторений и тип операции.

Проверка моста охватывает 1792 набора входов на каждом из четырёх исполнителей: native Crystal, raw MIR2, optimized MIR2 и Z80. Есть повторное использование специализации и конфликтующее имя параметра `meta_result`. Runtime-значение вместо compile-time count отвергается. Этот мост пока имеет узкий протокол; универсальный доступ метафункции ко всему исходному Crystal AST не реализован.

Следующая итерация добавила более полезный источник специализации - **раскладку целевой структуры**. В MetaArena поле `marker` находится по смещению 20 на 64-битном host и по смещению 7 на

Z80. Host-указатель занимает восемь байтов, целевой - два. Нельзя узнать offset у нативного Crystal и механически вставить его в программу Z80.

Импортёр сначала строит целевые типы. Затем настоящая Nanz-метафункция получает target offset, размер структуры и ширину указателя и генерирует чтение нужного байта. Тесты layout bridge сравнивают поведение, включая копию структуры и одноимённое поле двух разных типов. Конкретный marker-протокол пока ограничен; сам принцип уже проверен: машинные детали должны появляться после выбора машины.

DSSL, DSL и маленький язык из двух слов

Название «DSSL» требует оговорки: в этом срезе репозитория отдельная технология с таким именем не определена. Здесь точнее говорить **DSL**, domain-specific language, - небольшой язык для конкретной задачи. В MinZ такие задачи разные: описание hardware-операций, правила преобразований компилятора, схемы двоичных полей. У этих направлений разный статус; старые планы backend DSL не следует считать списком сегодняшних работающих возможностей.

Для двоичного потока уже есть маленький язык, который легко прочитать целиком:

```
import stream.bit_fields

@bit_fields("Header") {
    bits "kind" 3
    bits "flags" 5
}
```

Первые три старших бита - `kind`, следующие пять - `flags`. Реальная метафункция bit_fields проверяет имена и ширины, а затем создаёт структуру с байтовыми полями и функцию `Header_decode(raw:u16, dest:^Header)`.

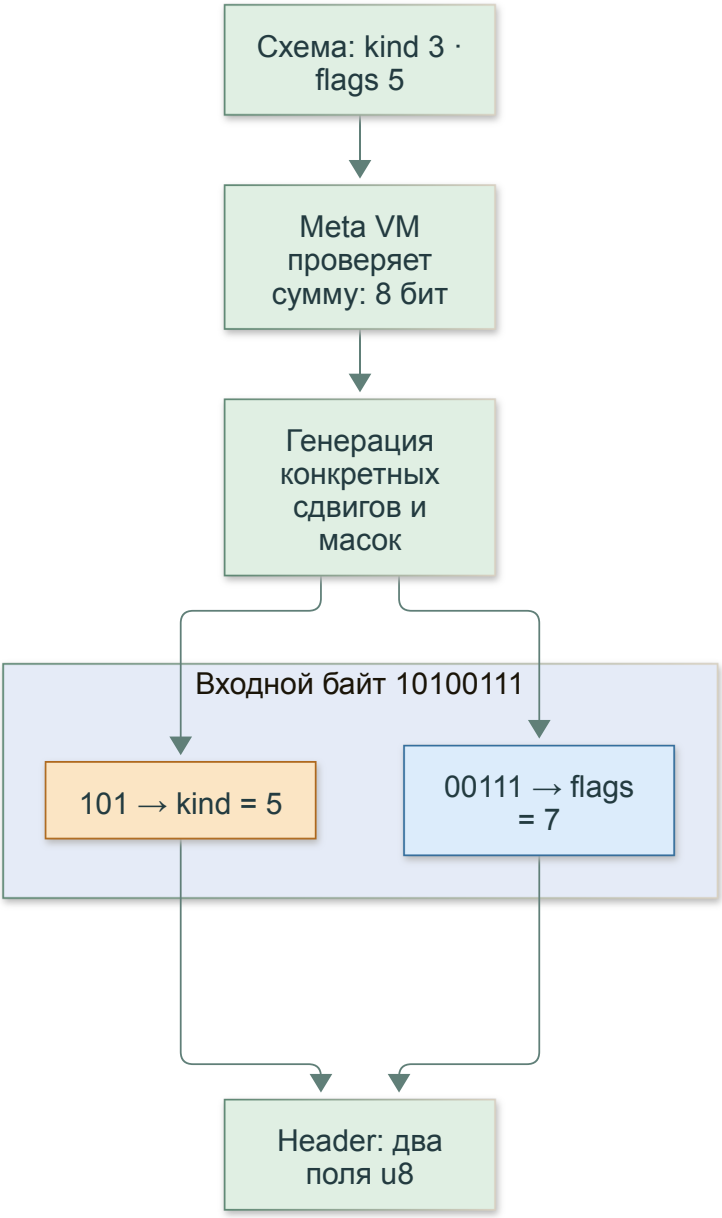


Схема 1.3 · На узком экране схему можно прокрутить горизонтально.

В этой схеме важна даже последняя коробка: результат занимает два байтовых поля, а не становится автоматически упакованным однобайтовым объектом. Мы описали разбор входного представления. Получившаяся структура выбирает удобное представление для последующей обработки.

Пример использования соответствует существующему тесту:

```
fun header_kind(raw:u8) -> u8 {
    var decoded:Header
    Header_decode(u16(raw), &decoded)
    return decoded.kind
}
```

Полная схема и функция выше образуют Nanz-пример. Сам механизм проверяется TestBitFieldMetaNative: все 256 входных байтов для двух полей, дополнительные 16-битные случаи и ошибочные схемы. Поддерживаются поля шириной 1-8 бит, не более восьми полей, общая ширина ровно 8 или 16 бит. Это небольшой реализованный контракт, который удобно расширять после новых проверок.

Что в этой идее от Erlang

В Erlang разбор битов давно является выразительной частью языка. Например, следующую строку можно выполнить в Erlang shell:

```
<<Kind:3, Flags:5, Length:16/little-unsigned-integer,
  Payload:Length/binary, Rest/binary>> = <<16#A7, 2, 0, 85, 204, 255>>.
```

Она связывает `Kind=5`, `Flags=7`, `Length=2`, выделяет двухбайтовый payload и оставшийся байт. Здесь декларация формы двоичных данных сразу участвует в сопоставлении. Размер следующего фрагмента может

зависеть от уже прочитанного поля. Это пример Erlang bit syntax; его правила описаны в официальном руководстве.

Битовый DSL MinZ берёт из этой идеи компактное описание полей. Полного Erlang bit syntax, произвольных bitstring patterns и подобного зависимого разбора в нём пока нет. Для переменной длины сейчас существует отдельный PacketDecoder, написанный обычным Nanz. Он разбирает пример протокола `prefix:u8, length:u16 little endian, payload:length` по частям.

Именно здесь заканчивается простота красивой строки. В файле данные могут закончиться посреди длины. Следующий кусок может прийти через секунду. Потребитель может ещё не обработать предыдущий пакет. Эти ситуации требуют сохраняемого состояния и правил владения буфером.

В MinZ они представлены явно. Это делает хороший следующий шаг для DSL вполне понятным: научить декларацию порождать проверенный автомат с теми же правилами. Пока такой общий генератор автоматов остаётся идеей; готовый код состоит из битовой метасхемы и переиспользуемых написанных вручную декодеров.

OTP полезно понимать до разговора об акторах

OTP - Open Telecom Platform - вырос из экосистемы Erlang и Ericsson. Это библиотеки, компоненты и принципы построения систем; Elixir использует эту уже существующую платформу. История и назначение кратко изложены на сайте Erlang/OTP, а её использование из Elixir - в официальном руководстве Mix и OTP.

Особенно полезна идея behaviours: отделить общий механизм сервера или автомата от обработчиков конкретной задачи. `gen_server`, `gen_statem` и supervisor формализуют разные роли. Дерево supervision описывает, кто наблюдает за рабочими процессами и что перезапускает при отказе. Эти понятия объясняет руководство OTP Design Principles.

Такое разделение вдохновляет и на маленькой машине. У декодеров разные форматы, но одинаковые вопросы: какой байт принят, есть ли готовое событие, кому сейчас принадлежат данные, как обозначается окончание, что происходит после ошибки?

Действующий SMF-декодер отвечает на них без фонового исполнения. Приложение вызывает `feed_byte`, получает результат и само решает, когда продолжать. Нет отдельного actor runtime, почтовых ящиков, вытесняющего scheduler или supervision tree. Состояние хранится в структуре, переходы выполняются обычными вызовами. Полезное сходство с OTP здесь относится к устройству интерфейса и явному протоколу компонента.

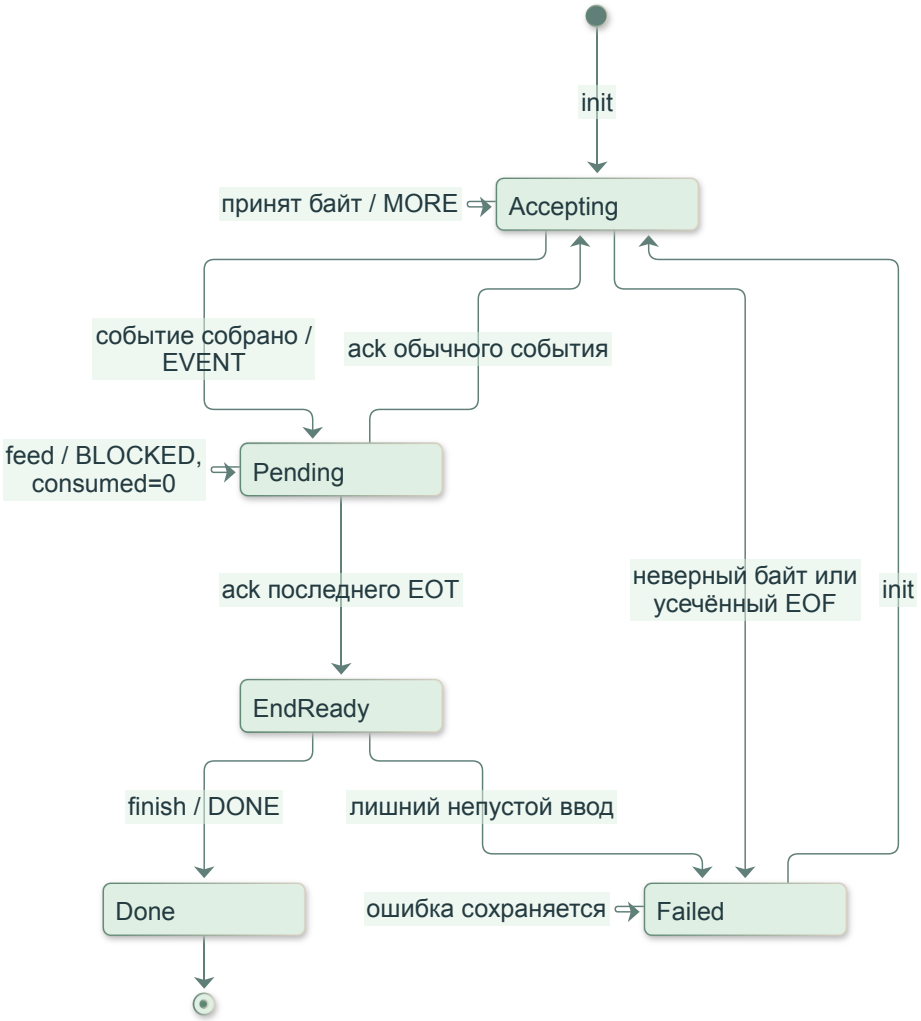


Схема 1.4 · На узком экране схему можно прокрутить горизонтально.

Это схема внешнего протокола, а не полный граф внутренних фаз SMF. Внутри есть чтение заголовка, длины дорожки, VLQ, статуса, данных и payload. Внешнему потребителю достаточно знать, можно ли подавать следующий байт и когда освобождать предыдущий результат.

Будущий кооперативный планировщик мог бы обслуживать несколько таких автоматов. Тогда потребуются отдельные решения: сколько работы разрешено за один шаг, как ограничены очереди, кто владеет буферами при переключении, где допустимы прерывания. Само наличие `feed` и структуры состояния ещё не отвечает на эти вопросы. Для текущего файлового преобразования явного синхронного управления уже достаточно.

Самая полезная операция - иногда «ещё не принимай байт»

В stdlib/midi/smf.nanz результат `feed_byte` упакован в `u16`: старший байт сообщает, принят ли входной байт, младший - статус. Если событие готово, оно сохраняется до `ack`. До подтверждения следующий ввод получает `BLOCKED` и `consumed=0`.

Так потребитель управляет темпом обработки, а декодеру не нужна растущая очередь. Meta/SysEx payload находится в предоставленном буфере. Пока событие выдано, его данные доступны потребителю; после `ack` буфер разрешено использовать для следующего события.

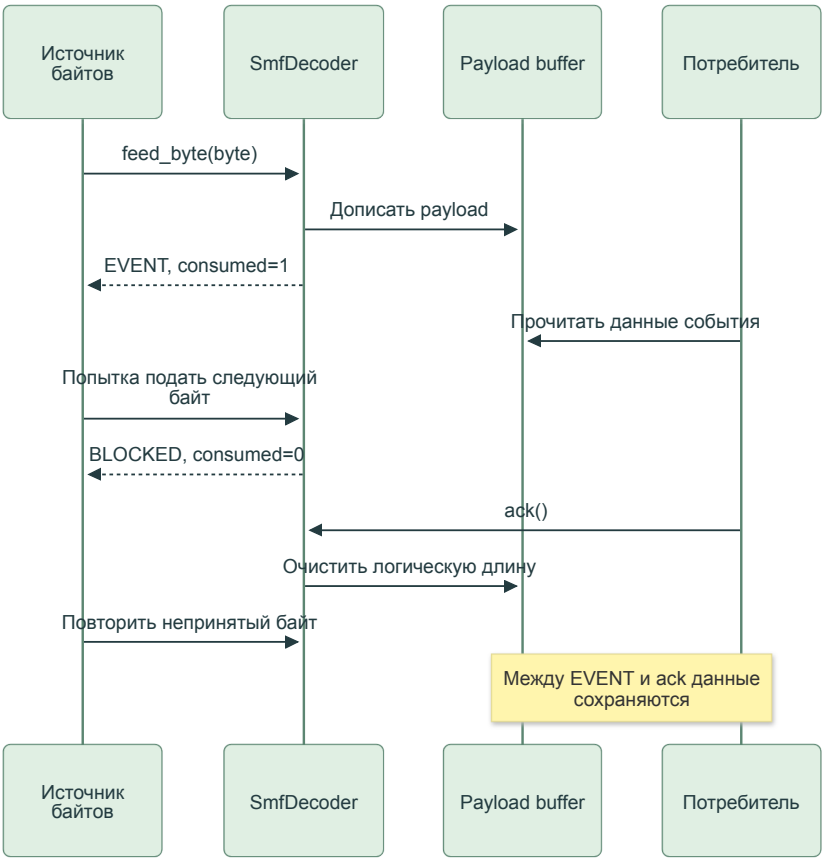


Схема 1.5 · На узком экране схему можно прокрутить горизонтально.

Это протокол временного пользования буфером, а не доказательство lifetime на уровне системы типов. Вызывающий код обязан держать дескрипторы и backing storage живыми и не накладывать их на неподходящие области памяти. Тем не менее явный срок действия ссылки гораздо легче проверить, чем неоговорённое «этот указатель пока работает».

Важна и разница между пустым куском и концом файла. Пустой кусок ничего не сообщает о будущем. Для EOF существует `finish()`: он подтверждает завершение после EOT всех дорожек или выдаёт ошибку усечения. Иначе случайная пауза источника могла бы выглядеть как корректное окончание музыки.

В простом потребителе MIDI обработка события помещается в небольшой фрагмент:

```
let result:u16 = parser.feed_byte(byte,&message)
let status:u8 = u8(result & 255)
if status == SmfStatus.EVENT {
    let kind:u8 = message.status & 240
    if kind == 144 {
        if message.data2 != 0 { note_ons = note_ons + 1 }
        else { note_offs = note_offs + 1 }
    }
    if kind == 128 { note_offs = note_offs + 1 }
    last_tick = message.tick
    parser.ack()
}
```

Это часть реальной функции `midi_byte`; её глобальные счётчики и буферы объявлены в том же файле. Нулевую velocity потребитель трактует как note-off. Декодер при этом сохраняет исходный wire status. Разделение позволяет независимо проверить точность разбора и музыкальную интерпретацию.

События SMF поступают в порядке дорожек файла. Их слияние в общий хронологический поток для воспроизведения - отдельная задача. Наличие `tick` и успешного парсинга не означает готовый MIDI-плеер.

Где заканчивается красивая идея и начинается доказательство

У базового бинарного эксперимента уже был серьёзный внешний корпус: четыре MIDI-файла, 28 724 события, сравнение всех полей и полного payload в raw MIR2, optimized MIR2 и Z80. На каждом событии проверялось поведение при неподтверждённом выходе. Отдельные тесты перебирали разбиения короткого потока на куски, перенос кольцевого буфера, EOF и повреждённые заголовки.

Следующая MIDI pairing-веха добавила музыкальное состояние: FIFO для одинаковой высоты на одном канале, разделение каналов, стабильное удаление из активного списка, закрытие в EOT и квантизацию. Её 38 случаев исполняются с кусками 1, 7 и 256 байтов на трёх целевых вариантах - 342 исполнения. Вместе с результатами независимо заданы ожидаемые ошибки, число принятых байтов, событий, источников и активных нот.

Затем появились файлы и две `.COM`-программы. Здесь доказательство меняет форму: уже недостаточно правильно вычислить массив в памяти. Нужно сохранить формат, закрыть файл, распознать усечение, не спутать физическое заполнение последней записи CP/M с дополнительными данными и не принять частичный результат после ошибки I/O.

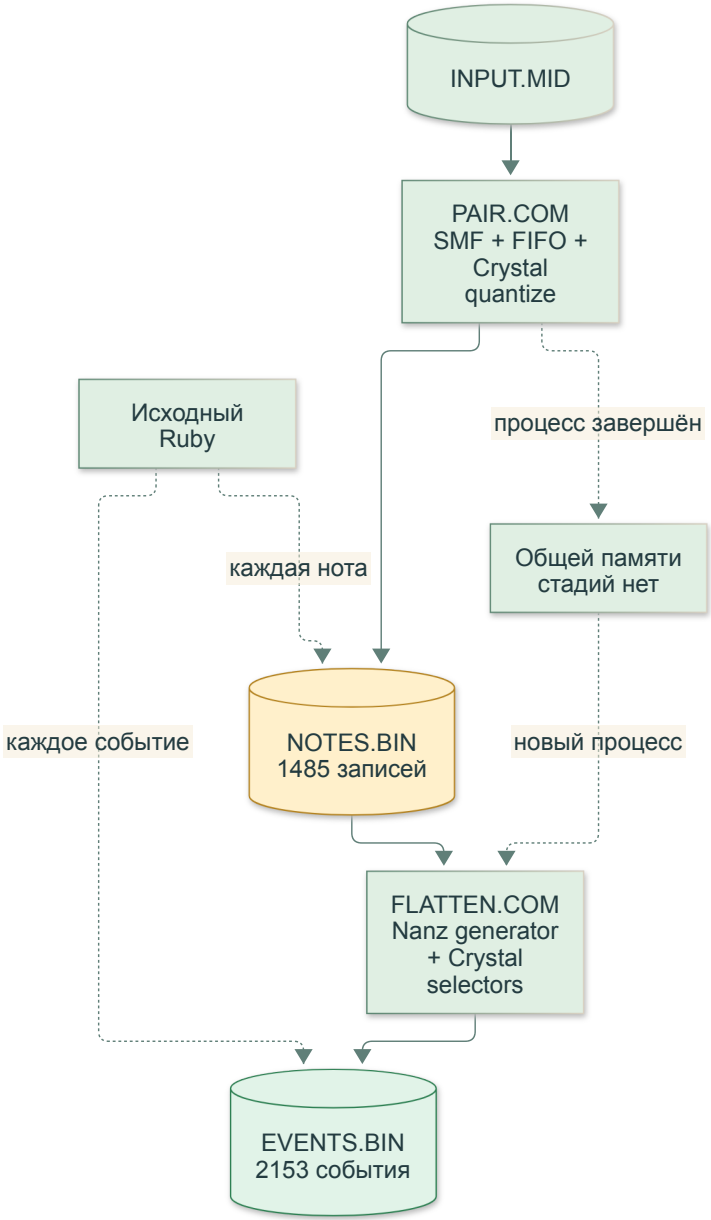


Схема 1.6 · На узком экране схему можно прокрутить горизонтально.

Оба образа включают полную арену нот размером 12 288 байт: это учтённая память. `PAIR.COM` занимает 30 914 байт загруженного образа, `FLATTEN.COM` - 22 750. Эти две величины не нужно складывать как одновременно живущую программу: стадии последовательно заменяют друг друга. Выходные события записываются через 128-байтовый буфер, а не накапливаются целиком.

В этой же точке полезно остановить воображение раньше, чем оно превратит эксперимент в готовый продукт. Это CP/M-цепочка в эмуляторе, с конкретным профилем Chronos и ещё ограниченной конфигурацией. Обработка аранжировки, echo/mix и конечный VT2/VTX не завершены. Измеренные пики стека относятся к выполненному корпусу; они не являются автоматически доказанной верхней границей для любой будущей программы.

На момент написания в отдельном незакоммиченном Spectrum-прототипе локально прошли 60 случаев на трёх вариантах исполнения - 180 проверок первой стадии. Там исследуются TRD, секторный ввод и страничная память. Эти результаты ещё не входят в данный срез `main` и не являются испытанием на физическом Spectrum. Подробности платформенного пути относятся ко второй статье; языковая архитектура от этого разделения только выигрывает: алгоритм, файловый адаптер и способ доступа к памяти можно проверять по отдельности.

Что можно попробовать самому

Все команды ниже запускаются из корня репозитория. Это существующие проверки, а не результаты нового прогона при написании статьи.

Битовые схемы и двоичные автоматы доступны без Ruby-корпуса:

```
go -C minzc test ./pkg/nanz \
  -run
'^(TestBitFieldMetaNative|TestPacketDecoder.*Native|TestSMF.*Native)$' \
  -count=1 -v
```

Мост макросов требует установленного Crystal с библиотекой компилятора, Go и Python:

```
python3 research/crystal-hir/verify_meta.py
python3 research/crystal-hir/verify_meta_records.py
```

Замороженный MIDI-корпус воспроизводит проверенные Ruby/Crystal-продукты без установки этих языков; fresh gate строит их заново и использует отдельный checkout Autosiril:

```
go -C minzc test ./cmd/autosiril-midi-native -count=1

python3 research/autosiril-native/midi/verify.py \
  --autosiril /path/to/autosiril --out /tmp/minz-native-midi-proof
```

Свежий прогон нужен, когда меняется смысл эталона или экспортируемой функции. Замороженный replay нужен, чтобы обычный CI не зависел от установленного Ruby, Crystal и внешних MIDI-файлов. Их связывают hashes исходников, входов и результатов; это разные способы поддерживать одно проверяемое утверждение.

У этой истории остаётся вполне конкретное продолжение. Можно дать автору протокола более выразительный язык состояний, сохранив текущие контракты `feed`, `ack`, EOF и ограниченного буфера. Можно расширить мост метапрограммирования, сохранив целевой layout и явные зависимости. Можно соединять следующие музыкальные стадии, измеряя полную память каждой.

Самый интересный результат уже виден в сегодняшнем коде: читабельный исходник и явный машинный бюджет становятся совместимыми, когда компилятор получает достаточно информации заранее. Ruby помогает удержать музыкальный смысл. Crystal разрешает типы и специализации. Nanz описывает память, протоколы и генерацию. А Z80 получает программу, у которой можно посчитать байты и проверить каждый переход на настоящих входных данных.

ДЕНЬ КОМПИЛЯТОРА · ГЛАВА 02

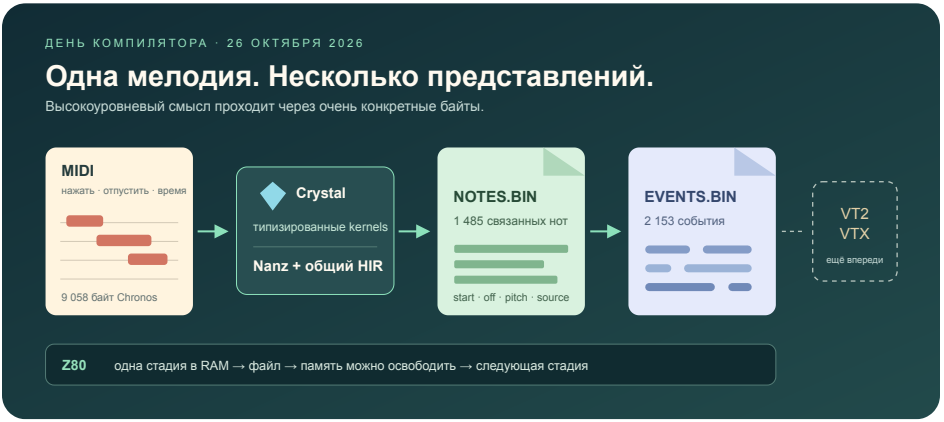
Мелодия между двумя файлами

Ленивость, генераторы, fusion, 64 байта RLE и нативный MIDI pipeline.

Как ленивость, генераторы и 64 байта RLE приблизили Autosiril к Z80

26 октября 2026 · День из жизни MinZ, часть 2

Все три статьи · Часть 1: языки и идеи · Часть 3: чему учиться у экосистем



У сегодняшнего дня есть довольно осязаемый результат: одну программу можно завершить, её память освободить, запустить другую - и мелодия продолжит своё путешествие.

Это не многозадачность и не спрятанный за красивым синтаксисом огромный runtime. Это две небольшие нативные утилиты, обычные файлы и Z80, который действительно исполняет полученный

машинный код в эмуляторе. Первая читает MIDI и связывает начала и окончания нот. Вторая читает записанные ноты и превращает их в события трёх каналов.

Но самое интересное произошло раньше, по дороге к этим двум файлам. Чтобы они стали возможны, пришлось изменить вопрос. Вместо «как уместить весь результат в память?» мы стали спрашивать: **«какую следующую вещь уже действительно попросил потребитель?»**

На этой маленькой перемене держится большая часть истории: ленивые вычисления, генераторы, fusion, аккуратное обращение с банковой памятью и неожиданно симпатичный RLE-декодер.

Сначала - честная фотография дня

На момент этой статьи в `main` уже вошли PR #262 и PR #263. Их самостоятельная файловая цепочка выглядит так:

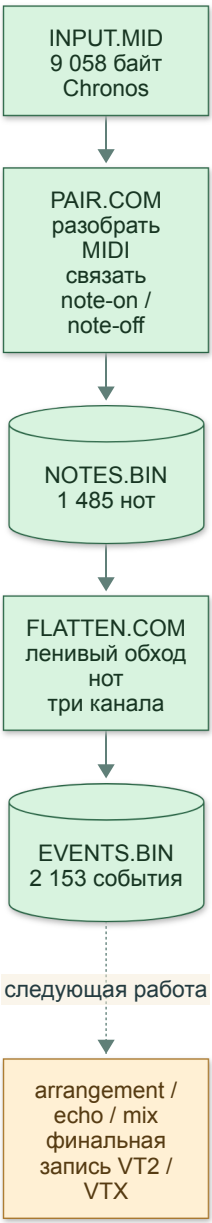


Схема 2.1 · На узком экране схему можно прокрутить горизонтально.

Утилиты запускаются по очереди в отдельных процессах `mze -t cpm`. Первая не передаёт второй заранее готовую арену из host-кода. После её завершения остаётся файл. Следующая утилита сама открывает его через BDOS, читает, проверяет и работает с данными на target.

Все **1485 нот и 2153 события** Chronos совпали с оригинальным Ruby Autosiril. Новая вторая стадия прошла **201 исполнение**: 67 случаев в raw MIR2, оптимизированной MIR2 и собранном Z80. Отдельно повторены прежние 342 MIDI-исполнения и 147 проверок файлового producer. Это число запусков, а не число песен: в корпусе есть границы ёмкостей, пустые входы, испорченные записи и ошибки ввода-вывода. Точный состав и receipts.

Полный нативный Autosiril пока не готов. Сегодняшний результат - проверенные первые стадии; последующие музыкальные преобразования и окончательный tracker-файл ещё предстоит довести до такого же состояния. Именно поэтому правая часть схемы выделена другим цветом.

Ruby умеет рассказывать, что мы хотим

Ruby прекрасен тем, что обработку коллекции легко записать почти как предложение. Например, такой учебный декодер раскрывает пары `[сколько, какое значение]`:

```
runs = [[3, 65], [0, 90], [2, 66]]
bytes = runs.flat_map { |count, value| [value] * count }
# => [65, 65, 65, 66, 66], то есть "AAABB"
```

Здесь хорошо виден смысл. Нулевой run ничего не добавляет, три буквы A сменяются двумя B. Но конкретно эта запись создаёт промежуточные массивы и готовит весь результат. Для настольного скрипта это часто совершенно разумная цена за ясность.

На Z80 нам хочется сохранить ясность, а стоимость выбрать иначе. Если потребитель просит только два байта, незачем сначала создавать пять. Если поток длиннее доступной памяти - весь результат вообще может не существовать одновременно.

Ruby тоже умеет выдавать элементы по запросу: `Enumerator` и `lazy` позволяют строить такие программы. Учебный вариант выше нужен для контраста двух способов выполнения, а не для противопоставления языков. И сам экспериментальный RLE ниже - отдельный тест компилятора; мы не утверждаем, что нашли именно такой `flat_map` внутри оригинального Autosiril.

На уровне замысла оба подхода говорят о той же последовательности. Различается момент, когда появляется очередной элемент.

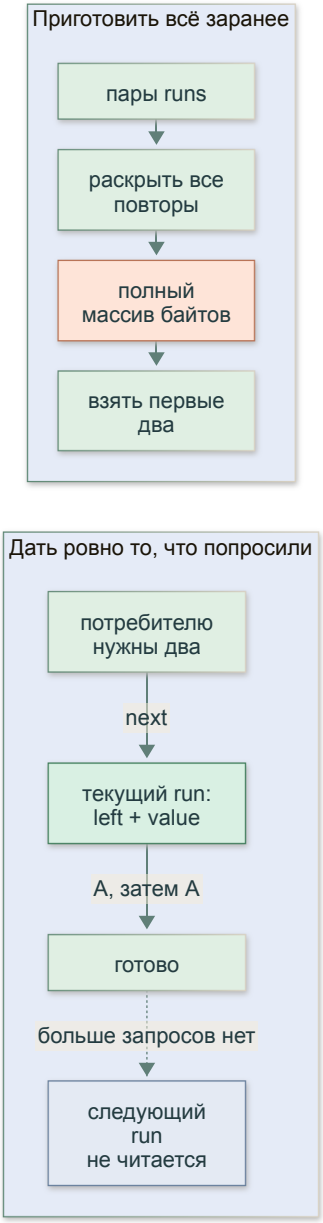


Схема 2.2 · На узком экране схему можно прокрутить горизонтально.

Генератор - маленькая машина с закладкой

В Nanz проверенный RLE producer записан так. Это отформатированная версия реального исходника измеряемого ядра:

```
gen runs(src:^u8, pairs:u8)->u8 {
    while pairs != 0 {
        var left:u8 = src[0]
        let value:u8 = src[1]
        src = src + 2
        pairs = pairs - 1
        while left != 0 {
            yield value
            left = left - 1
        }
    }
}
```

`yield` здесь означает: «вот следующий байт; продолжение пока подождёт». Нужно помнить, где остановились, сколько повторов осталось и какое значение выдавать. Для этого общее HIR lowering строит конкретный frame состояния и функцию `next`.

Отдельный стек для каждого генератора не требуется. Frame - данные; `next` - обычный вызов, который читает эти данные, проходит до следующего yield и сохраняет продолжение. Где именно хранить frame, решает окружающая программа и lowering. Это не обещание, что произвольный генератор всегда занимает несколько байтов: живые переменные и захваченные зависимости имеют реальную стоимость.

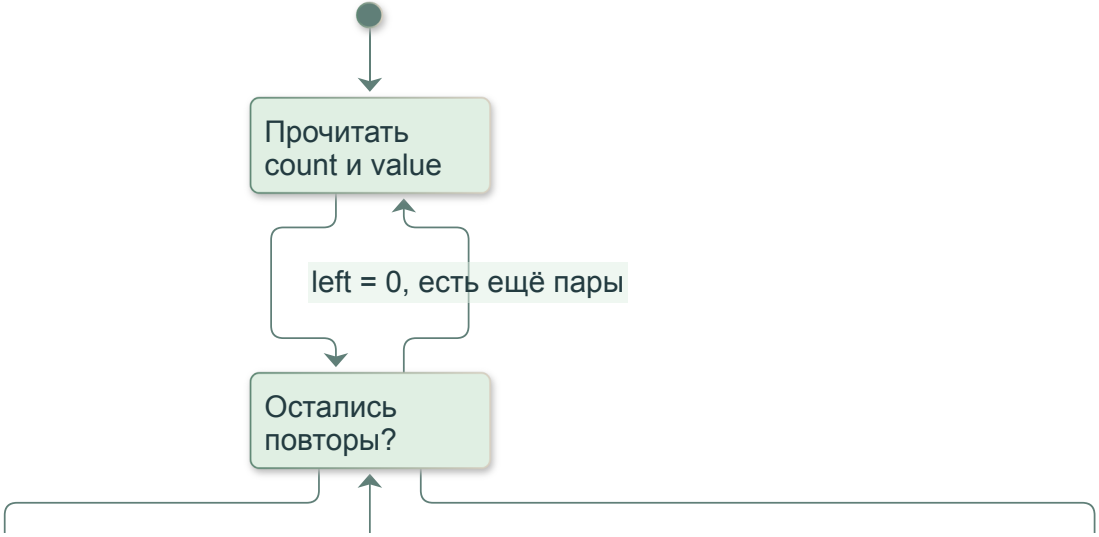


Схема 2.3 / часть 1 из 2

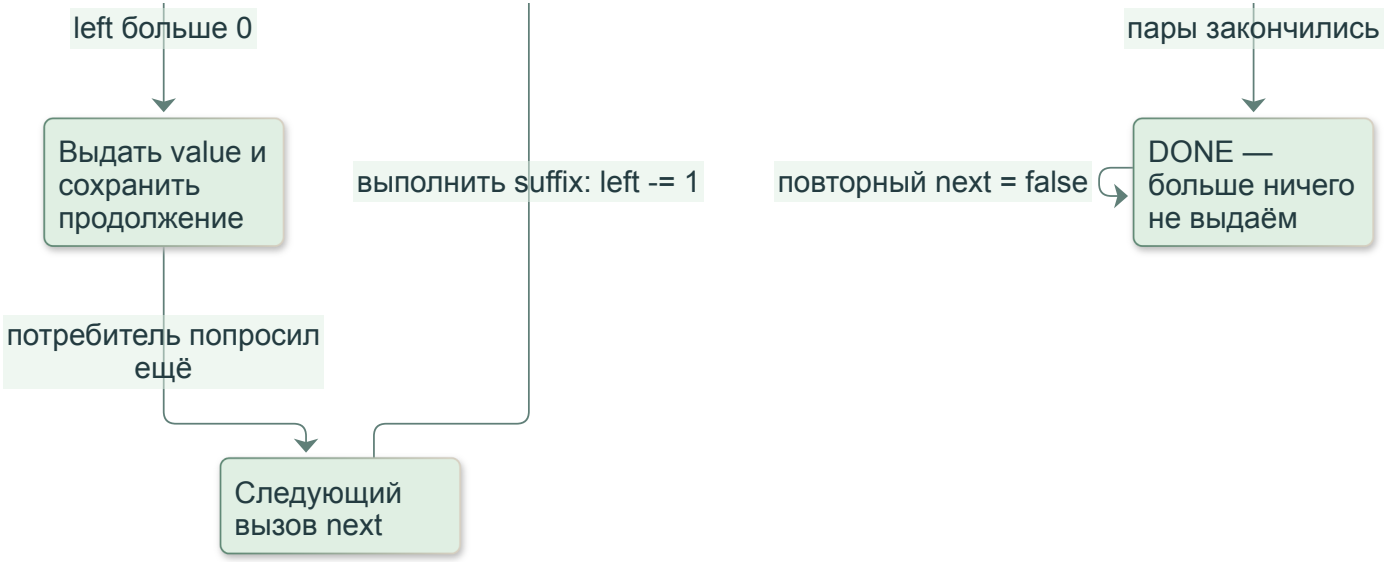


Схема 2.3 / часть 2 из 2

У Crystal уже есть узнаваемая запись той же идеи - метод с блоком и `yield`. Ниже сокращённая форма проверенного RLE-примера; полный корпус дополнительно ведёт trace-счётчики:

```
def rle_stream(src : Pointer(UInt8), pairs : UInt8, &)
  while pairs != 0_u8
    left = src[0_u16]
    value = src[1_u16]
    src = src + 2_u16
    pairs = pairs &- 1_u8
    while left != 0_u8
      yield value
      left = left &- 1_u8
    end
  end
  nil
end
```

Bridge переводит поддержанную форму Crystal producer в те же HIR `YieldStmt` и generator-функцию, которыми пользуется Nanz. Значит, frame lowering, исправления завершения и дальнейшие общие оптимизации не нужно придумывать второй раз для Crystal. Это пока проверенный subset одностороннего yield, а не импорт всей стандартной иерархии Crystal Iterator. Реальный пример и границы.

Ленивость - обещание о событиях, а не только о памяти

У ленивого producer есть более тонкая обязанность, чем экономить массив. Он не должен заранее выполнять работу, которую никто не запросил.

Пусть следующий run читается из аппаратного порта. Или переключает страницу памяти. Или содержит ошибку, до которой программа по своему смыслу вообще не доходит. Тогда лишний `next` - наблюдаемое изменение поведения.

Возьмём обычный Nanz consumer:

```
var accepted:u8 = 0
var total:u16 = 0
for value in runs(&data, 8) {
    if value == 0 { continue }
    total = total + u16(value)
    accepted = accepted + 1
    if accepted == 2 { break }
}
```

Мы просим два **принятых** значения. Нулевых значений producer может выдать больше; consumer их пропускает. Но после второго принятого элемента дальнейшие пары читать нельзя. Более того, `break` должен остановить producer до его следующего suffix. А `continue` consumer, наоборот, должен выполнить producer suffix и только потом запросить новый элемент. Иначе декодер может бесконечно выдавать один и тот же байт.

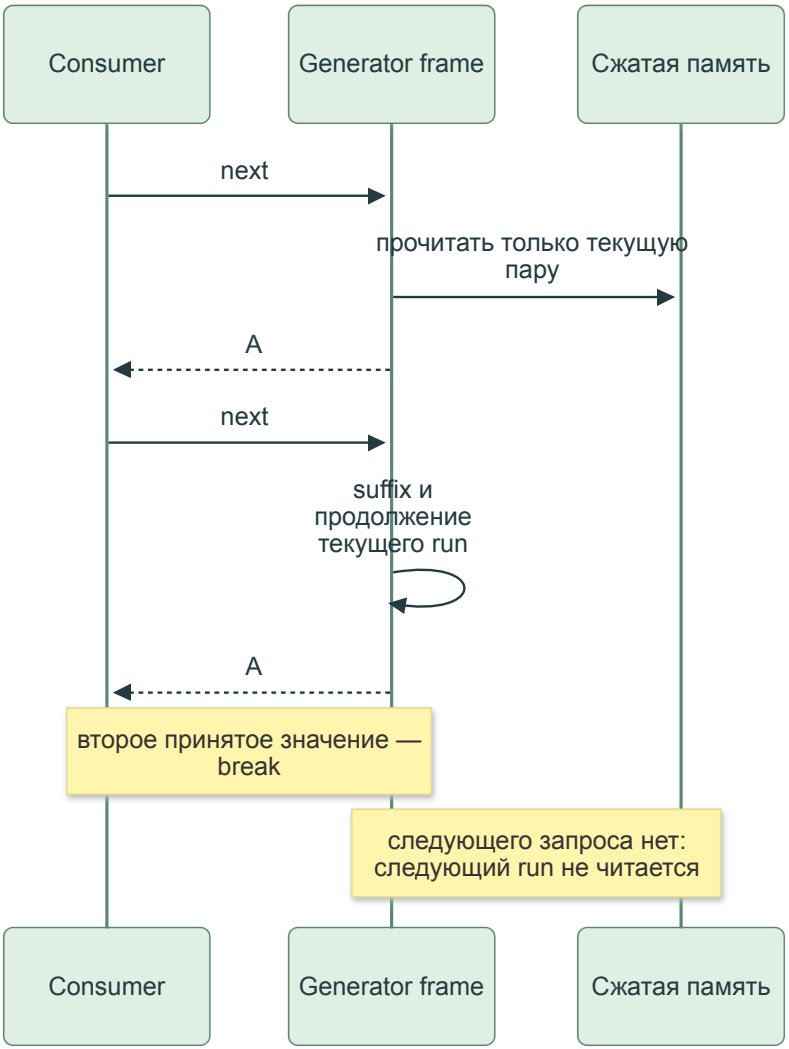


Схема 2.4 · На узком экране схему можно прокрутить горизонтально.

Именно поэтому наши тесты сравнивают не только сумму в конце. Они проверяют полную упорядоченную трассу: проверки условий, чтения, yield, тело consumer и suffix. Отдельные нативные read heatmaps подтверждают, что после раннего break последующие пары не были прочитаны. Это уже контракт вычисления. Семантические проверки RLE.

Fusion: убрать конвейерную ленту, сохранив порядок действий

Генератор с frame удобен, когда producer нужно сохранять между отдельными запросами. Но иногда весь маршрут виден компилятору: вот producer, вот единственный немедленный consumer.

Тогда вызовы `next`, запись выходного слота и часть состояния можно убрать. На месте yield оказывается тело consumer. Это и есть generator fusion в нашем ограниченном HIR-профиле.



Схематически сумма становится обычными вложенными циклами. Следующий фрагмент объясняет преобразование, а не претендует на дословный HIR dump:

```
while pairs != 0 {
    var left:u8 = src[0]
    let value:u8 = src[1]
    src = src + 2
    pairs = pairs - 1
    while left != 0 {
        total = total + u16(value) // consumer на месте yield
        left = left - 1
    }
}
```

Сложность спрятана в сохранении смысла `break`, `continue`, `return`, порядка вычисления аргументов и имён локальных переменных. Если consumer хочет выйти из всего producer, одного выхода из внутреннего `while` недостаточно: преобразование должно завершить и внешний. Если потребитель меняет backing memory, нельзя без доказательства заранее прочитать следующую пару.

Текущий pass поэтому допускает конкретные формы, включая этот двухуровневый RLE, и отказывает остальным. Он включается через `--fuse-generators`; это не общий автоматический fusion любой цепочки. Сложный Autosiril producer с ручным pull пока не сливается этим pass. Условия допуска и fallback.

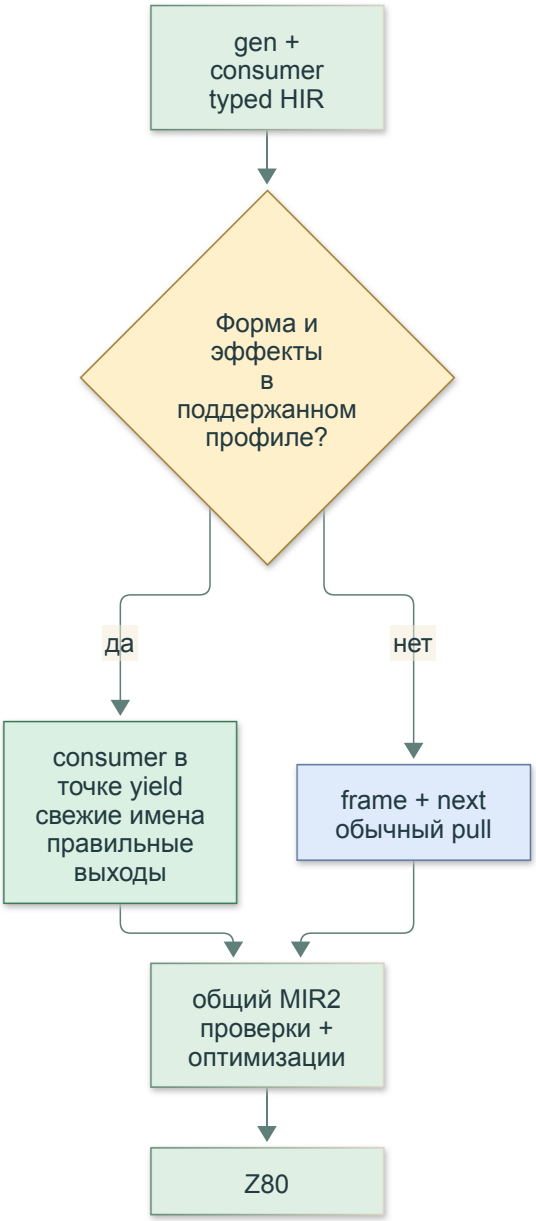


Схема 2.5 · На узком экране схему можно прокрутить горизонтально.

Ленивость отвечает на вопрос **«когда вычислять?»**. Fusion - **«нужны ли для этого отдельные вызовы и frame?»**. Можно иметь ленивый неслитый generator. Можно выполнить весь поток через слитые циклы. Экономия памяти и удаление накладных расходов связаны, но это разные решения.

Получилось ли красивое машинное продолжение?

На одном конкретном измеряемом RLE-ядре - да. Входные восемь пар загружаются во время исполнения; они раскрываются в 13 байтов с суммой 1362. Это не компиляция заранее известного ответа.

Вариант того же RLE-ядра	Код	Z80 T-states	Стек ниже входа
Обычный pull	311 байт	16 201	19 байт
Первый двухуровневый fusion	73 байта	3576	4 байта
Fusion + удаление мёртвой пересылки	**64 байта**	**3253**	**4 байта**

Все helper-функции входят в размер кода; только 16 входных байтов исключены. Последний вариант даёт полный linked binary 80 байт. Стек указан без начального двухбайтного адреса возврата. T-states измерены CPU-эмулятором, не физической платой. Это результат данного ядра, не обещание пятикратного ускорения любого Ruby-подобного кода. Измерения и исходники.

Последние девять байтов исчезли благодаря довольно забавной находке. В MIR2 три block-параметра пересылали друг другу значение, которым больше никто не пользовался. Каждый выглядел «использованным», потому что передавался соседу. Получился маленький бюрократический кружок: три кабинета передают друг другу пустую папку.

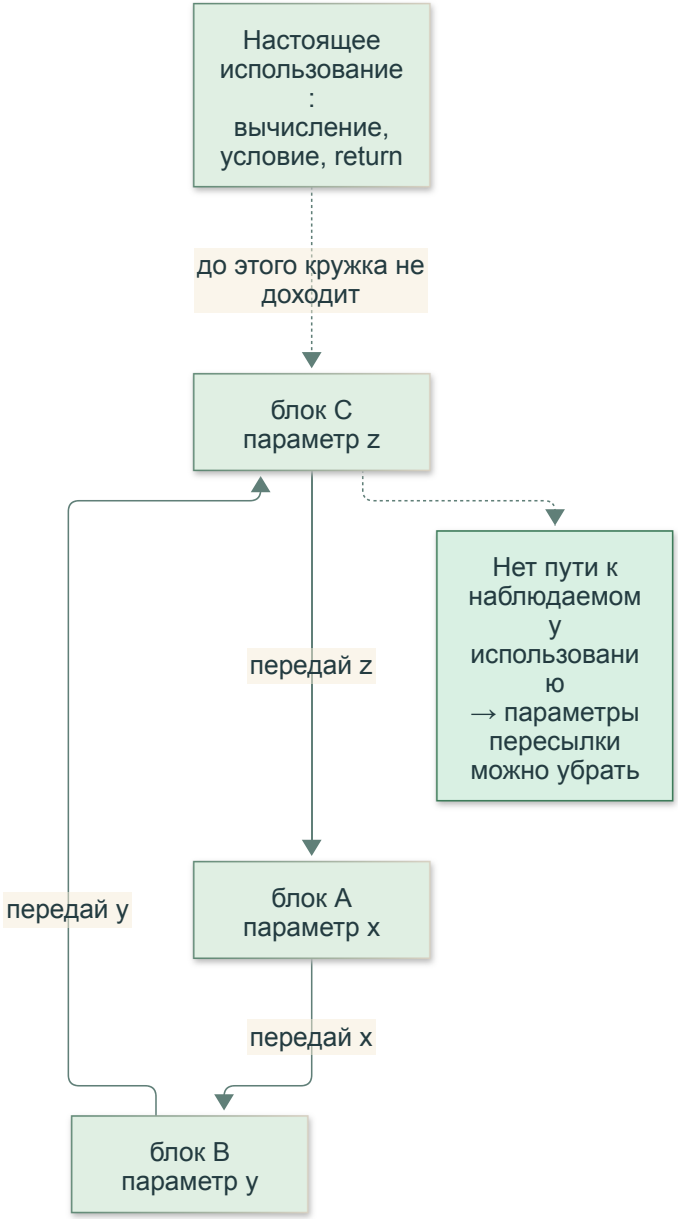


Схема 2.6 · На узком экране схему можно прокрутить горизонтально.

Анализ теперь начинает с настоящих использований и распространяет живость назад по переходам. Закрытый forwarding-only цикл без такого корня исчезает. Этот cleanup работает на общем MIR, а не только на RLE и не только на одном frontend. При этом он не получает права удалять чтения, вызовы, traps или записи просто потому, что их результат не нужен.

Так удобный генератор неожиданно помог найти улучшение, полезное всему компилятору. Вот более интересный смысл «обогатить HIR/MIR», чем добавить специальную инструкцию «быстро распакуй RLE».

А при чём здесь трансдьюсеры?

Трансдьюсер удобно понимать как преобразование шага обработки: «пропусти неподходящие значения», «измени принятое значение», «остановись после нужного числа результатов». В Clojure этот механизм отделён от конкретного источника и приёмника и позволяет композицию без промежуточных коллекций. Короткое определение и API.

У нас есть родственная инженерная цель: отделить смысл последовательности от того, лежит ли она в массиве, приходит из generator или читается с диска. Но текущий Nanz generator fusion не является реализацией Clojure transducer protocol. И fluent-запись `runs(...).filter(...).take(2)` в приведённом примере пока не добавлена: проверенный consumer записан явным циклом.

Это полезная граница. Нам уже доступен выигрыш от композиции и ограничения спроса; красивый универсальный API можно растить поверх доказанного поведения. Тогда синтаксис будет опираться на работающие правила остановки, а не скрывать неизвестный порядок эффектов.

MIDI - это ещё и бухгалтерия времени

RLE показывает идею на байтах. MIDI добавляет музыку и несколько видов времени.

Файл содержит события с delta-time, running status и границами дорожек. Note-on и note-off ещё нужно связать. Тики нужно превратить в строки. А затем определить, что конкретно должен делать tracker-канал в каждой строке: начать ноту, продолжить её, отпустить или выбрать одну из нескольких.

Поэтому **2984 MIDI-события, 1485 связанных нот и 2153 flatten-события - разные сущности**. Их числа не должны совпадать. Это не потерянная половина файла, а последовательные представления музыки.

Вот реальный Crystal kernel, которым пользуется Nanz-часть:

```
def crystal_cell_action(mask : UInt8) : UInt8
  return 0_u8 if mask == 0_u8
  return 0_u8 if mask == 2_u8
  return 4_u8 if mask == 4_u8
  1_u8
end
```

Биты означают start=1, continue=2, release=4. Продолжение без нового события не требует вывода, release-only выдаёт отпускание, остальные непустые комбинации приводят к запуску/перезапуску. Это сохранение поведения оригинального конвертера, а не новое правило MIDI. Все музыкальные kernels.

Nanz producer уже умеет выдавать очередную запись по запросу:

```
// Фрагмент реального flattened из lazy_flatten.nanz.
gen flattened(source:u8, mode:u8)->u16 {
    // ... найти следующую значимую строку и обновить active[] ...
    // ... получить action и выбрать pitch ...
    yielded = yielded + 1
    yield_row = row
    yield u16(pitch) | (u16(action) << 8)
    // ... убрать завершённые handles, сохранив порядок ...
}
```

Это иллюстративная вырезка, не самостоятельная компилируемая функция: полное тело существенно богаче. Оно хранит handles активных нот и двигается между значимыми границами. Ленивая выдача помогает не строить заранее весь выход; сами связанные входные ноты нынешняя нативная стадия всё ещё хранит в ограниченной арене.

Отсюда важное ограничение: «ленивый pipeline» не означает «в памяти никогда нет всей песни». Мы убираем материализацию там, где это уже проверено. Новое устройство остальных стадий ещё требует работы.

Два файла - способ сделать память снова свободной

У первого совместного in-process образа MIDI → pairing → три flatten-канала получилось **40 626 байт**. Это полезный работающий маршрут: никакой host не передаёт готовые ноты между фазами. Но несколько следующих стадий уже не обязаны помещаться в ту же картину памяти.

Здесь особенно хороша предложенная Alice Unix-подобная мысль: пусть каждый инструмент делает одну стадию, а граница между ними будет явной. Для этого не нужны Unix pipes или одновременное выполнение процессов. На CP/M команды могут просто запускаться по очереди.

Утилита	Полный образ	Из него globals	Статические spills	Максимум стека ниже входа
PAIR.COM	30 914 байт	15 063	440	66 байт
FLATTEN.COM	22 750 байт	14 380	165	63 байта

Оба образа уже включают резерв арены 12 288 байт. Эти два размера не складываются при последовательном запуске: первая программа закончилась, прежде чем загрузили вторую. Стек приведён отдельно и не включает начальный адрес возврата. Полная раскладка.

Файл между стадиями стоит дополнительных обращений к диску. Зато код, временные таблицы и локальное состояние предыдущей программы больше не обязаны занимать RAM следующей. В будущем часть маршрута можно заменить resident dispatcher/overlay или прямым вызовом, когда доказаны их правила загрузки и lifetime. Общая форма записей позволяет менять это решение без переписывания музыкальных правил.

Конец файла оказался интереснее, чем ожидалось

В CP/M физическая запись занимает 128 байт. Полезные данные редко заканчиваются точно на её границе. Поэтому бинарному pipeline нужен собственный логический конец.

Для ASIR v1 мы записываем заголовок с версией и шириной payload, DATA-записи и завершение:

```
ASIR | version | kind | width | flags | metadata | reserved
01   | полезная запись
01   | полезная запись
...
FF   | count:u32 little endian | "END!"
00... физическое заполнение последнего блока
```

Ненулевой END и подпись после count нужны не ради красоты. Первоначальный нулевой terminator у пустого потока нельзя было надёжно отличить от нулевого заполнения. Теперь потерянное завершение не превращается в удачный пустой результат только потому, что сектор дополнен нулями.

Нельзя также считать любой `0x1A` текстовым EOF: внутри MIDI это может быть обычный байт. Для SMF логический конец определяется самим форматом; CP/M-адаптер отдельно проверяет остаток последнего блока. В inter-stage файле логический END и padding тоже проверяются отдельно. Дополнительная целая физическая запись не считается безобидным заполнением.

И ещё один неожиданно музыкальный край: ровно 1024 выходных события и «было бы 1025, но буфер кончился» - разные результаты. Утилита проверяет дополнительный pull. В первом случае она завершает generator и пишет корректный END; во втором возвращает ошибку переполнения вместо успешной усечённой песни.

Почему всё это особенно к месту на Z80

Дело не в том, что Z80 внезапно получил большой компьютер внутри. Мы стали лучше описывать работу, которая ему действительно нужна.

Генератор хранит продолжение вместо полного результата. Fusion убирает отдельный протокол там, где его существование уже не нужно. Скалярные handles переживают смену банков лучше, чем случайно

сохранённый near pointer. Явный record format переживает смену утилиты лучше, чем дамп native struct. Ограниченный buffer позволяет отказаться от неограниченного роста heap - при условии, что его переполнение действительно проверяется.

С такими средствами в восьмидесятые было бы удобнее описывать распаковку картинок, чтение бинарных протоколов, проходы по нотам, выборку кадров или выдачу очередного тайла. Не потому, что исчезают ограничения: становятся видны места, где программисту действительно нужно решить вопрос памяти, порядка и остановки.

Сегодня CP/M-цепочка уже в `main`. Отдельный локальный Spectrum/Scorpion эксперимент успел пройти 180 запусков первого MIDI → TRD этапа, включая ошибки секторов и границы каталога; это ещё экспериментальная ветка, не интегрированная здесь поддержка physical TR-DOS ROM. Остальные музыкальные стадии остаются следующим большим участком пути.

А самый приятный итог дня, пожалуй, такой: запись программы становится ближе к тому, как мы о ней думаем. «Дай следующую ноту». «Пропусти пустое». «Этого достаточно». Компилятор учится превращать эти просьбы в обычные циклы, память и переходы - и иногда оставляет от всей красивой конструкции всего 64 байта.

Где потрогать руками

Начать можно с небольшого RLE gate; для музыкального gate нужен локальный checkout оригинального Autosiril и Ruby с его зависимостями:

```
python3 research/generator-fusion/dead-params/verify.py \
  --out /tmp/minz-rle-proof

PYTHONDONTWRITEBYTECODE=1 python3 research/autosiril-native/file-
flatten/verify.py \
  --autosiril /path/to/autosiril --out /tmp/minz-file-chain
```

Ссылки для дальнейшего чтения: ограниченный HIR fusion, RLE и спрос, удаление мёртвой пересылки, Crystal yield → общий generator, связанные файлы и все границы, отдельный in-process путь, ADR 0053 о последовательных утилитах.

Чемодан без виртуальной машины: что MinZ стоит привезти из LLVM, .NET и BEAM

Что взять у LLVM, Cranelift, Grace, C#/.NET, JVM/Kotlin, Rust и BEAM.

День компилятора, 26 октября 2026. Третья статья: идеи больших экосистем, которые помещаются в маленький компьютер.

Представим, что разработчик MinZ отправляется на конференцию языков программирования. LLVM предлагает ему строгий промежуточный язык. Cranelift - аккуратный инструмент для описания машинных правил. C# показывает работу с памятью без обязательной копии. Kotlin достаёт генератор состояний. Erlang объясняет, как пережить отказ одного компонента.

На обратном пути возникает проблема: у нашего компьютера может быть 64 килобайта адресного пространства. Весь сувенирный магазин не увезти.

Но привезти можно другое: способ думать о программе. Проверки, выполняемые на машине разработчика, не обязаны превращаться в инструкции на Z80. Удобная конструкция языка не обязана тащить сборщик мусора. Даже идея из многозадачной виртуальной машины может превратиться в обычный `switch`, несколько байтов состояния и ясный договор между функциями.

У MinZ уже есть подходящий фундамент: общий HIR, MIR2, интерпретатор MIR2, Z80 backend и независимые проверки собранного машинного кода. А последние эксперименты с Crystal, генераторами и музыкальным конвейером дают возможность обсуждать заимствования

на настоящих программах. Ниже **«работает» означает конкретный реализованный путь; «предлагаем» - следующий архитектурный шаг**. Срез репозитория для статьи - `a716ff184`.

Сначала разберёмся, кто вообще приехал на конференцию

В одном разговоре легко назвать LLVM, JVM и BEAM «виртуальными машинами». Для архитектуры компилятора такое сокращение опасно.

LLVM - компиляторная инфраструктура с собственным IR. Cranelift - backend, принимающий IR и выпускающий машинный код; Wasmtime использует его и для JIT, и для AOT. CLR и JVM относятся к средам исполнения. Kotlin и C# - языки со своими компиляторами и библиотечными экосистемами. BEAM - среда исполнения Erlang, а OTP добавляет организационные принципы и библиотеки. О Cranelift, LLVM IR.

Grace в этой статье - **собственная инфраструктура переписывания MinZ**, а не одноимённый язык и не отдельная машина исполнения. Она сопоставляет структурные образцы, проверяет условия и вызывает преобразования. Её место - внутри компилятора.



Схема 3.1 · На узком экране схему можно прокрутить горизонтально.

Это главный способ экономии: выполнить сложную работу заранее. Анализ времени жизни может остаться на host. Отбор специализации - тоже. На target попадёт выбранная программа и только та поддержка, которую требует её контракт.

Сегодня обычный путь MinZ - HIR → MIR2 → Z80. VIR служит офлайн-исследованиям; возвращать удалённый LIR ради красивой диаграммы не требуется. Эту границу фиксируют обзор компилятора и ADR о переносе полезных преобразований.

LLVM: одолжить точность договора

Самая ценная черта хорошего IR - возможность объяснить, почему преобразование законно.

Рассмотрим учебный пример:

```
x = read_port()
y = x * 0
return y
```

Умножение можно заменить нулём. Вызов `read_port()` исчезать не должен: он может подтверждать аппаратное событие. Другой пример:

```
a = load(p)
select_bank(3)
b = load(p)
```

Одинаковый численный адрес здесь ещё не означает одинаковый объект. Между чтениями могла измениться видимая память.

LLVM документирует SSA, требования к корректному IR и правила `volatile`; синтаксически допустимая инструкция не обязательно образует корректную программу. Для MinZ полезен именно подход: у

каждого преобразования должны быть явные предпосылки. Переносить весь набор LLVM-семантики, включая его сложные правила неопределённых значений, только ради сходства незачем. LLVM Language Reference.

В MinZ это уже имеет материальную форму. В HIR facts существуют `ReadsMemory`, `WritesMemory`, `DoesIO`, `MayTrap`, `UnknownEffect`; неизвестный внешний вызов остаётся консервативно неизвестным. В MIR2 instruction есть `Volatile` и информация о последовательных байтовых доступах. Слово «консервативно» означает полезную вещь: если компилятор не знает достаточно, он сохраняет операцию.

Предлагаемое развитие - уточнять эффекты там, где это открывает измеримую оптимизацию. Например, отдельно обозначать смену банка, сброс арены, доступ к конкретному региону. Тогда запрет на перенос чтения через переключение памяти станет свойством операции, а не негласным знанием автора оптимизатора.

Не нужен сразу идеальный анализ всех указателей. Нужен первый контракт, который закрывает реальный контрпример и позволяет безопасно ускорить соседний случай.

SSA уже приехал; теперь надо беречь его паспорт

SSA даёт каждому вычисленному значению одно определение. Анализ def-use связывает это определение с местами использования. Отсюда возникают практические вопросы: действительно ли результат больше никому не нужен? Можно ли перенести инструкцию? Какие значения должны пережить вызов?

На слиянии ветвей MIR2 использует **параметры блоков**, а не phi-инструкции. Это уже видно в коде, а не только в плане. Учебная запись:

```
left:  jump join(7)
right: jump join(9)
join(value: u8):
    return value
```

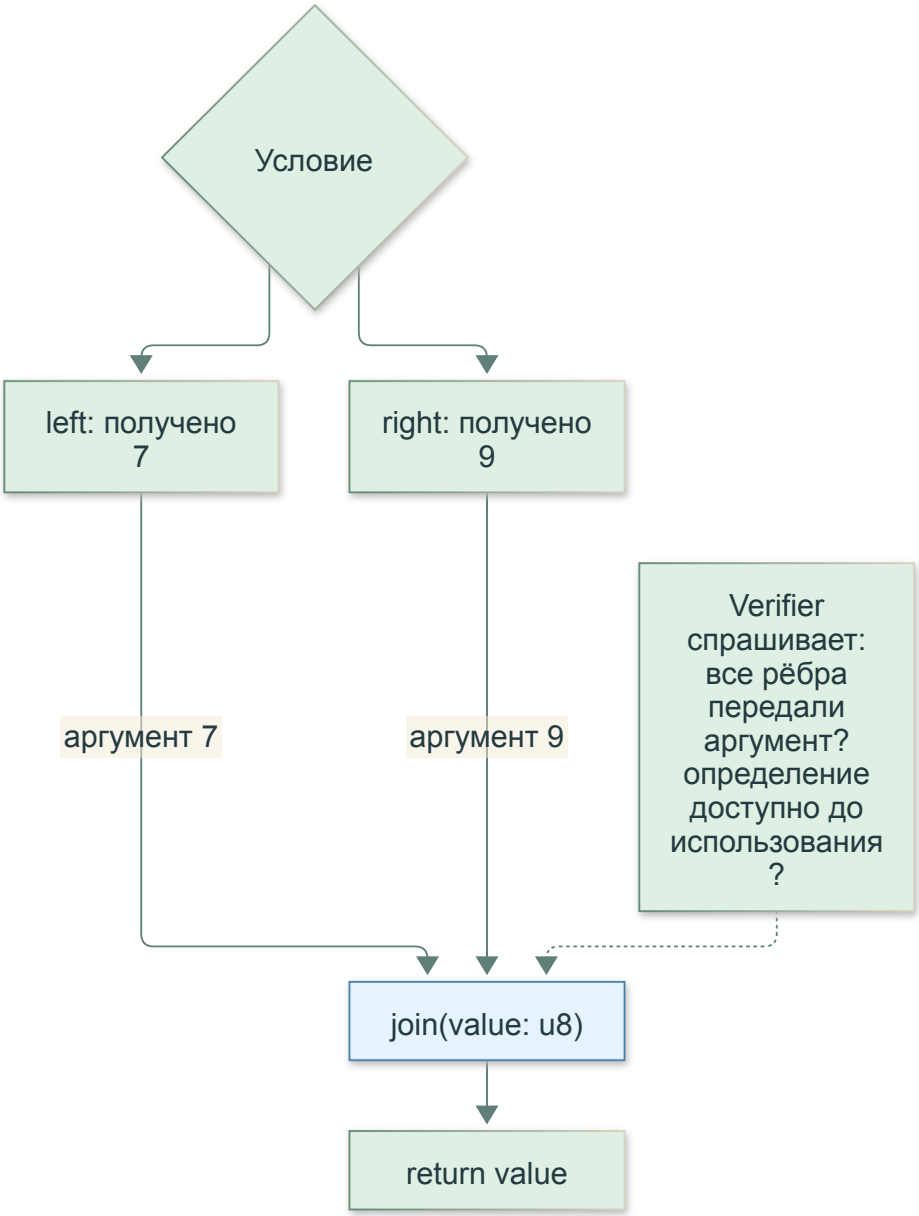


Схема 3.2 · На узком экране схему можно прокрутить горизонтально.

В MLIR значения также могут приходить через аргументы блоков. Заимствовать полезно дисциплину работы с этой формой: изменение параметра требует согласованного изменения входящих рёбер. MLIR Language Reference.

MIR2 verifier уже проверяет доминирование определений, завершённость блоков и число аргументов на рёбрах. Но он не выполняет универсальную строгую проверку типов каждого ребра: некоторые числовые преобразования сохраняют существующий VM-контракт. HIR verifier тоже прямо ограничивает свои обещания: структурная корректность не доказывает эквивалентность, безопасность захватов или возврат на всех путях.

Именно такое описание границ хочется перенять у зрелых инфраструктур. Verifier - турникет на входе в следующий проход. Он не заменяет доказательство, что поезд едет туда же.

Cranelift и Grace: правила должны быть читаемыми, а разрешения - строгими

ISLE в Cranelift - типизированный язык правил lowering. Правила компилируются в код, сопоставляющий образцы; общие части сопоставления могут разделять работу. Это хороший пример того, как вынести знание об инструкциях из гигантской россыпи условных операторов. ISLE в репозитории Bytecode Alliance.

У MinZ есть собственный механизм. Вот настоящее правило из constant_if.grace:

```
(grace hir-constant-if 10
  (match (block ?b (term-kind "hir.block")))
  (where (hir-has-constant-if ?b))
  (action (custom "hirSelectBranch" ?b)))
```

Выглядит коротко, но важная часть находится за `where`. Адаптер допускает именно литеральное логическое условие; действие сохраняет структурный блок и лексические связи. Нельзя взять произвольное «условие, которое вроде всегда истинно» и выбросить его вычисление вместе с побочными эффектами.

HIR Grace работает с бюджетом переписываний; после изменения проверяются HIR и пониженный MIR2. Сложные изменения выполняет Go-код. MIR2 Grace описывает CFG-образцы и специальные предикаты. Поэтому декларативная запись делает правило обозримым, но не превращает реализацию действия в автоматически доказанную теорему.

Что стоит перенять дальше? Разделить три вопроса, которые часто слипаются:

1. **Узнали ли мы форму?** Например, индексный проход по массиву.
2. **Разрешено ли преобразование?** Нет ли опасного alias, переполнения адреса, лишнего чтения или изменения спроса?
3. **Выгодно ли оно на этой машине?** Сколько осталось байтов, тактов, пересылок и сохранений?

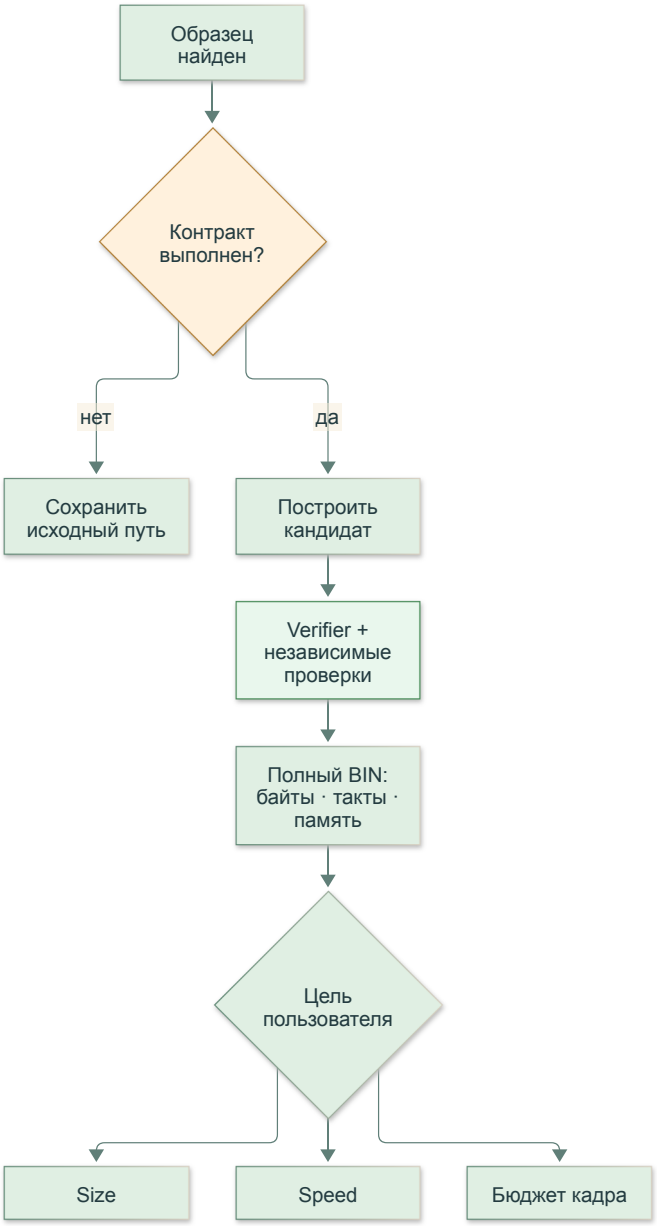


Схема 3.3 · На узком экране схему можно прокрутить горизонтально.

Это контур разработки и измеренного отбора кандидатов, а не утверждение, что обычная компиляция автоматически повторяет все независимые native-тесты.

На Z80 последнее особенно существенно. Удобное выражение может вынудить allocator сделать лишние пересылки. Два хороших локальных решения могут столкнуться за A или HL. Поэтому правило не должно само назначать себе медаль за уменьшение числа узлов IR.

Grace, PBQP и PFCCO решают разные задачи: преобразования, размещение значений и согласование вызовов. Проверенная таблица назначений - ещё один источник кандидатов. Их композиция оценивается на конечном коде, с ценой границ между частями. Это уже направление MinZ; заимствование из Cranelift помогает сделать его яснее, а не требует заменить весь backend.

C#/.NET: указатель с длиной - хороший билет, но не бессрочный

`Span<T>` полезен как урок устройства API: программа получает представление непрерывной памяти без обязательного выделения нового массива. В C# семейство `ref`, `ref struct` и правила безопасной области ограничивают, куда может уйти ссылка. `Span<T>` может представлять разные источники памяти; сам выбор такого типа не отменяет GC во всей программе и не делает произвольный внешний указатель безопасным. Microsoft: ref safety и работа с памятью.

Для MinZ это очень узнаваемая задача. Byte view можно представить как адрес и длину. Crystal bridge уже связан с общей Nanz-библиотекой Buffer/View, а кольцевой FIFO выдаёт два borrowed segments. В исследовательских проверках переносимого дескриптора различаются layout target и host: например, описанный borrowed view занимает 4 байта на проверенном Z80-пути и 16 байт в native host-модели. Итерация 7, общая byte-библиотека.

Но красивый дескриптор пока отвечает лишь на вопрос «где и сколько». Остаются другие:

- кто владеет памятью и когда её можно переиспользовать;
- может ли изменение владельца инвалидировать view;
- разрешён ли второй изменяющий доступ;
- может ли view пережить `yield` или смену банка.

Учебный псевдокод будущего API:

```
view = arena.borrow(handle)
yield view[0]
use(view[1])   // после возобновления это окно могло сменить банк
```

Здесь нет ошибки сложения адресов. Ошибка находится во времени.

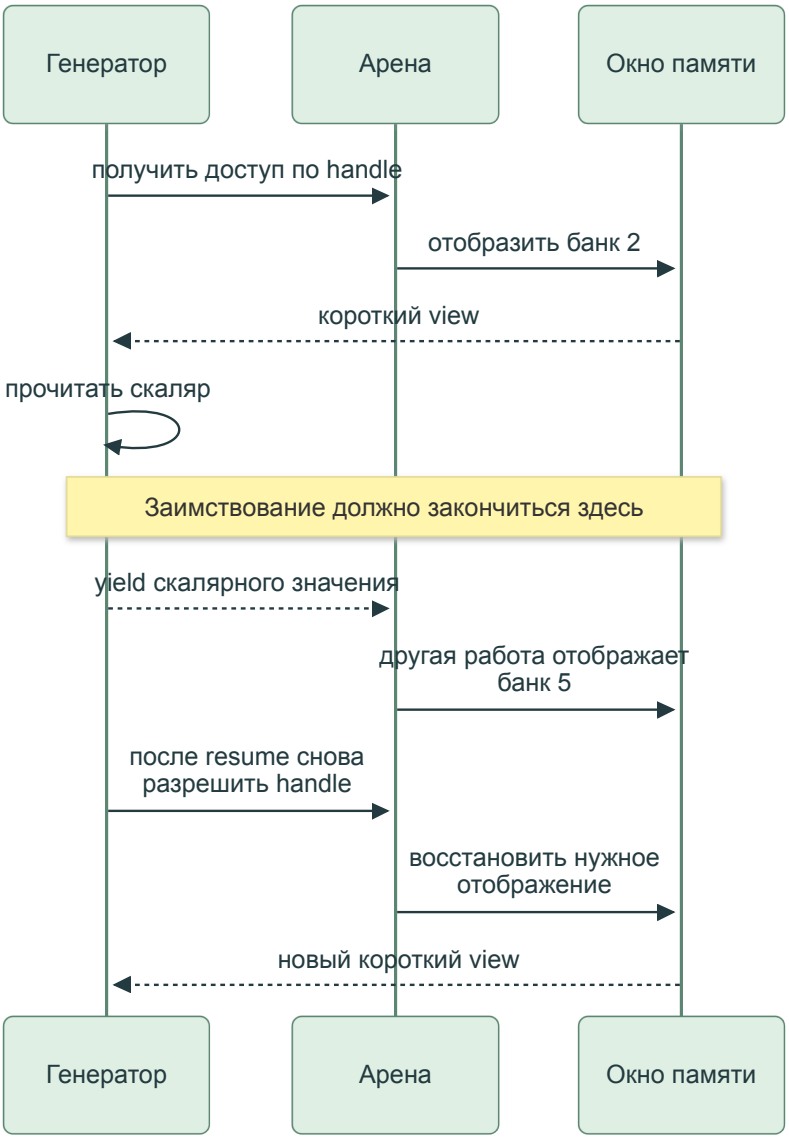


Схема 3.4 · На узком экране схему можно прокрутить горизонтально.

ADR 0051 предлагает именно такое разделение: долговечный handle и короткий доступ к отображённому фрагменту. Первое направление - библиотека и симулятор, не готовая общая система адресных пространств и не доказанная работа с аппаратными прерываниями.

Самая важная оговорка уже записана в ADR генераторов: нынешний G1 принимает некоторые сырые указатели на источник **без доказательства его времени жизни**. Ограничения на frame не доказывают, что память под указателем переживёт очередное возобновление. Называть это законченным borrow checker было бы ошибкой.

У Rust полезно позаимствовать требование обосновывать действительность ссылок и совместимость изменяющих доступов. Но символ `&` сам ничего не проверяет: нужны анализ происхождения, модель aliasing, обработка вызовов и понятная граница unsafe. Rust Book: references and borrowing.

Для MinZ разумный первый шаг - ограниченный проверяемый профиль: регион, известный владелец, короткое заимствование, запрет переноса через инвалидирующее действие. Такой профиль проще объяснить и испытать на контрпримерах, чем обещание «Rust для Z80».

JVM и Kotlin: контракт на входе, автомат состояний на выходе

У JVM есть ценный архитектурный урок: принимаемое представление проверяют до исполнения. Спецификация bytecode verification описывает ограничения на инструкции, типы и состояния исполнения. MinZ может развивать такую культуру вокруг HIR, импортов и ABI, не принимая формат `.class` и его объектную модель. JVM Specification, verification.

Особенно показателен Crystal bridge. В нём уже используется настоящий semantic frontend Crystal: нормализация, типизация, разрешение перегрузок, готовые специализации и раскрытие macro. Поддержанный результат затем экспортируется в общий HIR. Это гораздо содержательнее, чем «узнать похожий синтаксис», но область допуска всё равно ограничена. Semantic bridge, итерация 1.

Заимствование здесь - **принимать проверенную специализацию вместе с контрактом**. Типы аргументов, результаты, ошибки, layout и допустимые эффекты должны согласоваться с принимающей стороной. Мономорфизация сама по себе не стирает GC-зависимости произвольной программы. Можно принять закрытую scalar-функцию; из этого не следует перенос всей библиотеки Crystal.

У Kotlin второй полезный урок: приостанавливаемая функция превращается в автомат состояний с сохранёнными локальными значениями. Это описано в спецификации coroutine lowering. Kotlin: coroutine state machine.

В MinZ scalar `gen/yield` уже понижается в frame вызывающей стороны и обычную функцию `next(frame, out) -> bool`. Сохранять нужно значения, живые **через** точку приостановки. Временное значение, вычисленное и полностью использованное до `yield`, не обязано становиться постоянным полем frame.

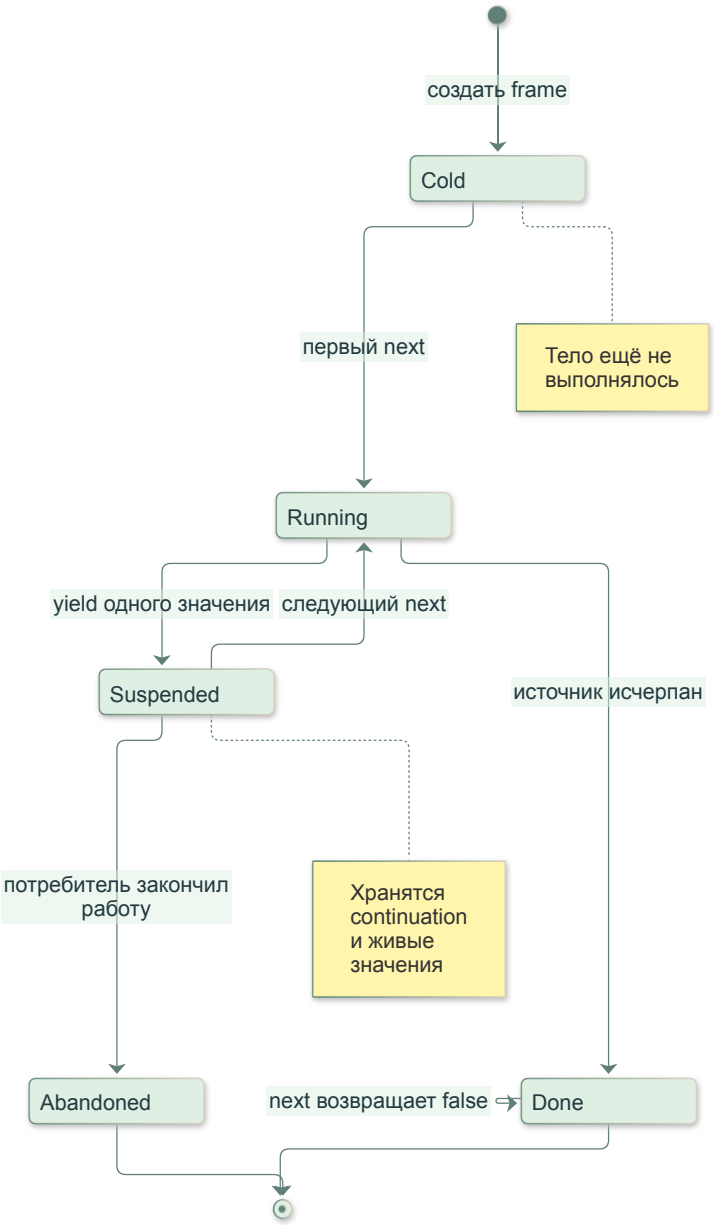


Схема 3.5 · На узком экране схему можно прокрутить горизонтально.

Диаграмма не подразумевает планировщик. `next` вызывается обычным кодом. Завершение сохраняется; отказ потребителя продолжать работу не должен исполнять оставшийся хвост производителя.

Сегодня есть и ограниченная opt-in fusion: `--fuse-generators` допускает проверенные формы непосредственного потребителя, включая описанный RLE-профиль. В его измеренном kernel 311 байт и 16 201 T превратились в 73 байта и 3 576 T. Это результат конкретного теста; ручные курсоры и неподдержанные формы сохраняют pull-путь. ADR 0050 и границы fusion.

Главная опасность fusion - нарушить спрос. Для `filter(x > 0).take(2)` над `[0, 0, 7, 8, 9]` потребитель должен получить `7, 8`; элемент `9` ему уже не нужен. Если чтение источника наблюдаемо, лишний запрос меняет программу, даже когда итоговый массив совпал. Поэтому оптимизировать нужно последовательность значений **и эффектов**.

Типы и формы: заимствовать способ исключать ошибки

Kotlin sealed-иерархии полезны возможностью знать закрытый набор вариантов и проверять полноту `when`. Для маленького target эта идея может жить в компактном теге и payload, без динамической иерархии объектов. Kotlin: sealed classes.

У Nanz уже есть enum/ADT и проверка случаев в поддержанном синтаксисе; актуальную форму надо смотреть в парсере. Старый ADR 0035 содержит более широкое проектное описание, которое нельзя автоматически считать реализованным layout всех ADT.

Полезное продолжение - точные типы протоколов. Например, источник данных может вернуть `Item`, `End` или `Error`. Живая пустая очередь требует отдельного `Pending`: временное отсутствие элемента не равно окончательному исчерпанию. Это архитектурный эскиз, не заявление о новом готовом API.

Другой вид информации - форма данных: длина фиксирована, шаг известен, набор вариантов закрыт, эффект выбран при компиляции. Такие факты позволяют статически специализировать алгоритм. Но каждое размножение специализаций съедает код. Для Z80 важно учитывать не только исчезнувший виртуальный вызов, но и десяток новых тел функций.

Хорошая система типов помогает выбрать маленькое представление. Хорошая политика специализации не позволяет этой экономии незаметно превратиться в переполненный BIN.

BEAM/OTP: взять жизненный цикл компонента

У OTP работники выполняют работу, supervisors следят за ними и могут перезапускать. Повторяющиеся схемы организации выделяются в behaviours. Это урок о границах ответственности, полезный далеко за пределами Erlang. OTP Design Principles.

Для MinZ интересен минимальный вопрос: что компоненту нужно, чтобы начать, обработать очередное событие, сообщить ошибку и закончить? Если ответы явные, его можно сделать обычным конечным автоматом с фиксированным состоянием.

Это особенно видно в уже выполненной CP/M-цепочке:

```
INPUT.MID --PAIR.COM--> NOTES.BIN --FLATTEN.COM--> EVENTS.BIN
```

Две независимые программы передают данные через настоящий файл. В проверенном Chronos-профиле 1 485 paired notes дают 2 153 события; результат сверяется с Ruby. Target исполняет музыкальную обработку, host предоставляет файловый BDOS I/O и проверку. Это ограниченные стадии, не весь MIDI → VT2 конвертер. Протокол и воспроизведение.

Связь с OTP здесь - наша архитектурная аналогия, а не перенос Erlang. У каждой стадии есть входной контракт, предел ёмкости, явное завершение и ошибки. Перезапуск такой утилиты мыслим потому, что промежуточный файл задаёт границу работы; безопасный повтор и атомарная публикация результата потребовали бы собственного протокола.

Переносить автоматически следом scheduler, процессы, почтовые ящики и GC не нужно. Даже на маленькой машине можно получить большую часть ясности от явных состояний и сообщений фиксированного формата. Но изоляция по соглашению функций не равна защищённой памяти процессов: ошибочная запись через сырой указатель по-прежнему может испортить соседа.

Что положить в чемодан первым

Самое выгодное заимствование - связать архитектурную идею с проверкой, которая умеет её опровергнуть.

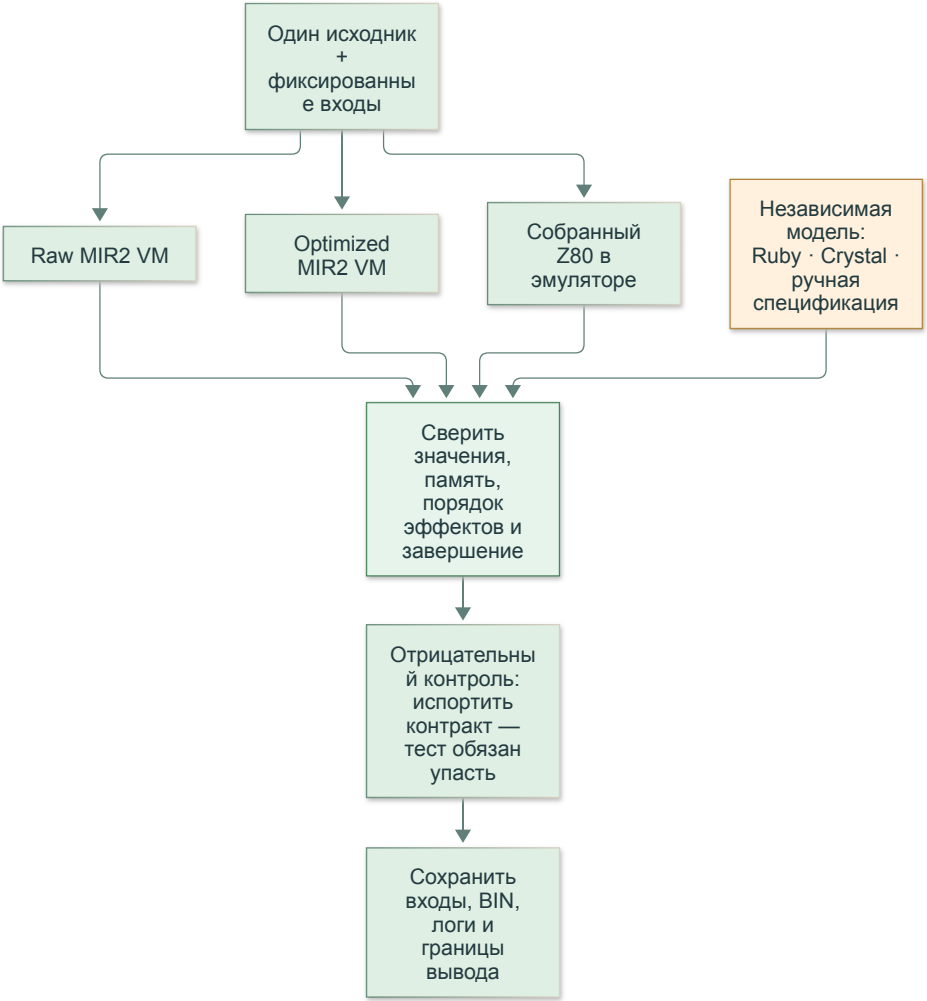


Схема 3.6 · На узком экране схему можно прокрутить горизонтально.

Здесь native означает исполнение собранных Z80-инструкций в эмуляторе. Физическое железо, его периферия и точное поведение прерываний требуют отдельной проверки.

Согласие двух реализаций полезно; независимое ожидание полезнее, когда обе могли унаследовать одну ошибку. А проверка неправильного варианта показывает, что сам стенд способен заметить проблему.

Для следующего этапа MinZ я бы выбрал такую последовательность:

Очередь	Заимствование	Конкретный выигрыш
1	LLVM/JVM: проверяемый контракт IR и импорта	Ошибка обнаруживается у границы, где возникла
2	C#/Rust: ограниченные правила времени жизни	View и generator frame не обещают больше, чем доказано
3	Cranelift: обозримые правила и строгие условия	Оптимизацию можно ревьюить по одному преобразованию
4	Kotlin: liveness и автоматы состояний	Frame хранит необходимое; fusion сохраняет спрос
5	OTP: явный жизненный цикл стадий	Ошибка и исчерпание становятся частью протокола

Эта таблица - предложение приоритетов, не объявление реализованного roadmap.

Полезная абстракция на восьмибитной машине проходит два экзамена. Сначала она должна объяснить автору, какую программу тот написал. Затем - объяснить компилятору, какие затраты можно убрать, а какие являются частью смысла.

В этом и состоит хороший багаж из больших экосистем: точные обещания, проверяемые границы, небольшие состояния и достаточно информации, чтобы сделать работу заранее. За столом остаются LLVM,

Cranelift, CLR, JVM и BEAM. На Z80 отправляется программа, которая знает, за что отвечает каждый её байт.